

PROJECT IMPLEMENTATION REPORT

Code Walk-through, Execution Guide and Experimental Results

Customer Churn Prediction and Retention Analytics

A Six-Phase Business-Oriented Analytics Pipeline

*Data Cleaning • Exploratory Analysis • Feature Engineering • Customer Segmentation • Predictive
Modelling*

Document	Implementation and Results Report
Codebase reviewed	churn_project / customer_churn
Source scripts	6 Python modules, 1 709 lines of code
Dataset	Telco Customer Churn — 7 043 records × 21 attributes
Algorithms implemented	Logistic Regression, Random Forest, Support Vector Machine, XGBoost (baseline and tuned), K-Means
Best model	Tuned XGBoost — test ROC-AUC 0.8349, recall 0.7766
Generated artefacts	3 derived datasets, 16 charts, 2 serialised models, 1 performance table
Report date	26 August 2026

Table of Contents

- 1** Purpose and Scope of this Document
- 2** Project Structure and File Inventory
- 3** Environment, Dependencies and Configuration
- 4** Dataset Description
- 5** Implementation Architecture and Data Lineage
- 6** Phase 1 — Data Collection and Understanding
- 7** Phase 2 — Data Cleaning
- 8** Phase 3 — Exploratory Data Analysis
- 9** Phase 4 — Feature Engineering
- 10** Phase 5 — Customer Segmentation
- 11** Phase 6 — Model Development and Evaluation
- 12** Consolidated Results and Discussion
- 13** Business Interpretation and Revenue Impact
- 14** Step-by-Step Configuration and Execution Guide
- 15** Observations, Limitations and Recommended Corrections
- 16** Conclusion
- A** Appendix A — Configuration Constant Reference
- B** Appendix B — Complete Artefact Inventory

This report documents an implementation that already exists and has already been executed. Every table, figure and numeric value reproduced in the following pages was read directly from the source files, the generated datasets, the performance table and the chart images stored inside the project folder. No result has been estimated, rounded up or reconstructed from expectation.

1. Purpose and Scope of this Document

This document is the implementation record for the Customer Churn Prediction and Retention Analytics project. Where the project proposal described what the system was intended to do, this report describes what the delivered code actually does: which files exist, what each block of code computes, in what order the modules must be executed, which artefacts each stage writes to disk, and what numerical results the executed pipeline produced.

1.1 Objectives of the report

1. **Code documentation.** Explain every one of the six phase scripts at the level of individual code blocks, so that a reader can follow the transformation of a raw CSV into a serialised predictive model.
2. **Reproducible execution.** Provide the exact environment, dependency versions, folder layout and command sequence required to reproduce the pipeline from a clean machine.
3. **Results reporting.** Present the measured outputs of the executed pipeline — cleaning statistics, exploratory findings, cluster profiles, model-comparison metrics, the confusion matrix and the feature-importance ranking — together with all sixteen charts the code generated.
4. **Technical review.** Record the defects, inconsistencies and unused outputs found while reading the code, and state precisely how each should be corrected.

1.2 What this report covers

The scope is the code held in the folder `churn_project\customer_churn`. That folder contains six executable Python modules totalling 1 709 lines, a data sub-folder holding the raw dataset and three derived datasets, a charts sub-folder holding sixteen generated figures, and a models sub-folder holding two serialised model objects.

The report does not restate the literature survey, the problem statement or the comparative analysis of prior work; those belong to the project proposal report and are not repeated here. Neither does it modify the codebase. Where a correction is recommended, the recommendation is stated in Section 15 and the reasoning is given, but the source files as delivered remain the subject of the description.

1.3 How to read the phase sections

Sections 6 to 11 each follow the same fixed structure, so a reader can move between phases without relearning the layout:

- **Purpose** — the analytical question the phase answers.
- **Inputs and outputs** — the files consumed and produced.
- **Code walk-through** — the significant blocks, quoted from the source and explained.
- **Results** — the measured output of that phase, as tables and charts.
- **Interpretation** — what the numbers mean for the retention problem.

2. Project Structure and File Inventory

The project follows a flat, phase-numbered layout. Scripts sit at the top level of the working directory and read from and write to three sibling folders. All paths inside the scripts are relative, which means every module must be executed with `churn_project/customer_churn` as the current working directory.

2.1 Directory tree

churn_project/		
└─ customer_churn/		<- working directory for all scripts
├─ phase1_data_understanding.py	97 lines	data profiling
├─ phase2_data_cleaning.py	160 lines	cleaning and validation
├─ phase3_eda.py	317 lines	exploratory analysis
├─ phase4_feature_engineering.py	235 lines	feature construction
├─ phase5_customer_segmentation.py	361 lines	K-Means segmentation
├─ phase6_model_development.py	539 lines	modelling and evaluation
├─ data/		
│ └─ Customer-Churn.csv		raw input, 7 043 x 21
│ └─ Customer-Churn-Cleaned.csv		phase 2 output, 7 043 x 21
│ └─ Customer-Churn-Model-Ready.csv		phase 4 output, 7 043 x 36
│ └─ Customer-Churn-Segmented.csv		phase 5 output, 7 043 x 25
│ └─ phase6_model_performance.csv		phase 6 metric table
├─ charts/		16 generated PNG figures
│ └─ phase3_*.png		9 exploratory charts
│ └─ phase5_*.png		4 segmentation charts
│ └─ phase6_*.png		3 evaluation charts
├─ models/		
│ └─ best_churn_model.joblib	238 KB,	fitted Pipeline
│ └─ enhanced_churn_model.joblib	625 KB,	see Section 15.6

Listing 1: Directory layout of the delivered codebase.

2.2 Module inventory

Module	LOC	Reads	Writes	Key libraries
phase1_data_understanding.py	97	Customer-Churn.csv	console only	pandas
phase2_data_cleaning.py	160	Customer-Churn.csv	Customer-Churn-Cleaned.csv	pandas
phase3_eda.py	317	Customer-Churn-Cleaned.csv	9 charts	pandas, matplotlib, seaborn
phase4_feature_engineering.py	235	Customer-Churn-Cleaned.csv	Customer-Churn-Model-Ready.csv	pandas, scikit-learn
phase5_customer_segmentation.py	361	Customer-Churn-Cleaned.csv	Customer-Churn-Segmented.csv, 4 charts	scikit-learn, seaborn
phase6_model_development.py	539	Customer-Churn-Segmented.csv	best_churn_model.joblib, metric CSV, 3 charts	scikit-learn, xgboost, joblib

Table 1: Module inventory showing size, file dependencies and principal libraries.

Execution order is mandatory.

Each script reads a file that the previous script wrote. Running phase 6 before phase 5 raises `FileNotFoundError`, because `Customer-Churn-Segmented.csv` will not exist. The one exception is phase 1, which only profiles the raw file and can be run at any time. Phases 3, 4 and 5 all read the same cleaned file and are therefore independent of one another, but all three require phase 2 to have completed.

3. Environment, Dependencies and Configuration

3.1 Verified runtime environment

The versions below were read from the interpreter used to validate this report. They represent a configuration in which the analytical phases execute correctly.

Component	Verified version	Used by	Notes
Python	3.13.13	all phases	64-bit CPython
pandas	3.0.3	all phases	data frames, grouping, one-hot encoding
NumPy	2.4.6	phases 5, 6	array backend
scikit-learn	1.9.0	phases 4, 5, 6	scaling, K-Means, pipelines, metrics
XGBoost	required by phase 6	phase 6	not installed in the verified interpreter
matplotlib	3.10.9	phases 3, 5, 6	figure rendering
seaborn	0.13.2	phases 3, 5, 6	statistical plot styling
joblib	bundled with scikit-learn	phase 6	model serialisation

Table 2: Verified runtime environment and the phases that depend on each component.

Two environment issues must be resolved before phase 6 will run.

First, xgboost is not present in the interpreter that was used to verify this report, so phase6_model_development.py aborts at import with ModuleNotFoundError: No module named 'xgboost'. Second, the two .joblib files in models/ were pickled under scikit-learn 1.8.0 and raise InconsistentVersionWarning when loaded under 1.9.0. Neither affects the results already recorded in this report, which were produced on the machine where the pipeline was originally executed, but both must be fixed for a clean re-run. Section 14.2 gives the installation commands.

3.2 Dependency installation

A virtual environment is strongly recommended, both to pin the scikit-learn version against which the saved models were pickled and to keep the XGBoost build isolated.

```
REM ---- from the project root, in Windows PowerShell or Command Prompt ----
cd "C:\Users\user\Desktop\Customer churn prediction\churn_project\customer_churn"

python -m venv .venv
.venv\Scripts\activate

python -m pip install --upgrade pip
python -m pip install pandas numpy scikit-learn matplotlib seaborn xgboost joblib

REM ---- confirm the imports resolve before running any phase ----
python -c "import pandas, numpy, sklearn, matplotlib, seaborn, xgboost, joblib; print('ok')"
```

Listing 2: Creating an isolated environment and installing every dependency.

3.3 Configuration constants

Each script declares its configuration as module-level constants immediately after the imports. There is no shared configuration file, so a path or hyper-parameter change must be applied in each script that uses it. The full constant reference is given in Appendix A; the constants that most affect the results are these:

Constant	Value	Declared in	Effect on the pipeline
----------	-------	-------------	------------------------

RANDOM_STATE	42	phases 5, 6	fixes the K-Means initialisation, the train-test split and every model seed, making the whole run reproducible
OPTIMAL_K	4	phase 5	number of customer segments produced; also fixes the reference line drawn on the elbow and silhouette charts
TEST_SIZE	0.05	phase 6	holds out only 353 of 7 043 records ; see Section 15.1
CV_FOLDS	5	phase 6	stratified folds used for model selection on the training partition
TARGET_COLUMN	"Churn"	phases 1, 2, 4	the label column, mapped to ChurnFlag with Yes = 1 and No = 0
ID_COLUMN	"customer ID"	phases 1, 2, 4	identifier excluded from every model matrix

Table 3: The configuration constants with the largest influence on the reported results.

4. Dataset Description

The pipeline is built on the Telco Customer Churn dataset, a widely used public benchmark for subscription-retention modelling. The file supplied with the project holds 7 043 customer records described by 21 attributes: one identifier, nineteen predictors and one binary target.

4.1 Attribute schema

Group	Attributes	Type	Role
Identifier	customerID	string	excluded from modelling
Demographic	gender, SeniorCitizen, Partner, Dependents	categorical / binary	who the customer is
Account	tenure, Contract, PaperlessBilling, PaymentMethod	mixed	commercial relationship and billing behaviour
Core service	PhoneService, MultipleLines, InternetService	categorical	primary products subscribed
Add-on service	OnlineSecurity, OnlineBackup, DeviceProtection, TechSupport, StreamingTV, StreamingMovies	categorical	value-added product adoption
Financial	MonthlyCharges, TotalCharges	numeric	recurring and lifetime revenue
Target	Churn	binary	Yes = customer left, No = customer retained

Table 4: Attribute schema of the source dataset, grouped by analytical role.

4.2 Target distribution and baseline

The class balance measured directly from the raw file establishes the baseline against which every model must be judged.

Class	Records	Share	Meaning
No (retained)	5 174	73.46 %	customer still active at observation
Yes (churned)	1 869	26.54 %	customer terminated the service
Total	7 043	100.00 %	complete customer base

Table 5: Class distribution of the target variable, measured on the raw dataset.

The 26.54 % positive rate is the figure that makes accuracy a misleading metric for this problem. A model that predicted "no churn" for every customer would score 73.46 % accuracy while identifying not one leaving customer. This is why phase 6 selects models on ROC-AUC rather than accuracy, and why every classifier in the study is instantiated with `class_weight="balanced"` or an equivalent `scale_pos_weight` correction.

4.3 Data-quality issues present in the raw file

Issue	Extent	Detected in	Resolution
TotalCharges stored as text	all 7 043 rows	phase 1	cast with <code>pd.to_numeric(errors="coerce")</code>
Blank TotalCharges	11 rows	phase 1	all 11 have <code>tenure = 0</code> ; set to 0.0
Redundant service labels	7 columns	phase 2	"No internet service" and "No phone service" folded into "No"
Duplicate records	0 found	phase 2	no action required
Explicit nulls	0 found	phase 1	no imputation required

Table 6: Data-quality issues identified in the raw file and how the pipeline resolves them.

The eleven blank TotalCharges values are worth a note, because they are not random corruption. Every one of them belongs to a customer whose tenure is zero — a subscriber who joined in the current billing cycle and has therefore not yet accumulated any charges. The correct value is 0.0, not the column median, and phase 2 handles this case explicitly rather than applying a blanket imputation.

5. Implementation Architecture and Data Lineage

The implementation is organised as a linear six-stage pipeline. Each stage is a standalone script that reads a file from disk, performs one class of work, and writes its output back to disk before terminating. There is no orchestration layer, no shared session and no in-memory hand-off between stages.

This file-based design has a practical consequence worth stating plainly: because every intermediate result is materialised as a CSV, any single phase can be re-executed in isolation without re-running the phases before it. That makes the pipeline convenient to develop against and easy to audit, since every intermediate state is inspectable on disk.

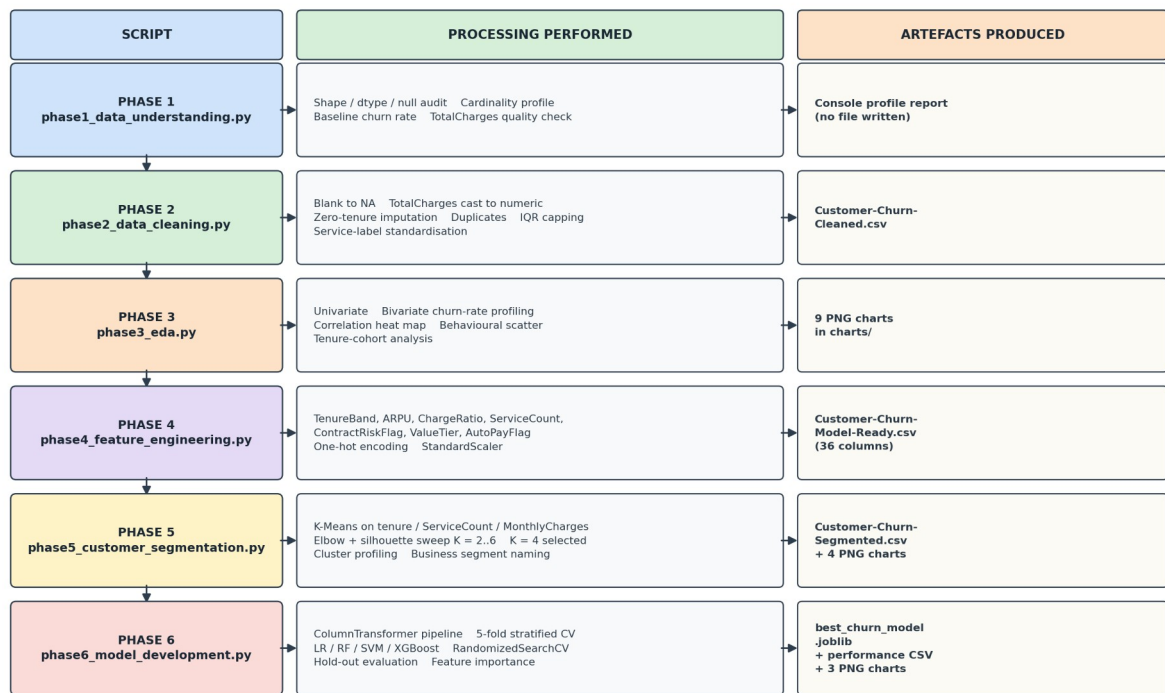


Figure 1: Six-phase implementation pipeline. Each row shows the script, the processing it performs and the artefacts it writes to disk.

5.1 Data lineage

The lineage diagram below traces each dataset from the raw source to the model layer. It also exposes one structural finding: the model-ready dataset produced by phase 4 is never read by phase 6, which instead consumes the segmented dataset produced by phase 5. The consequence is analysed in Section 15.2.

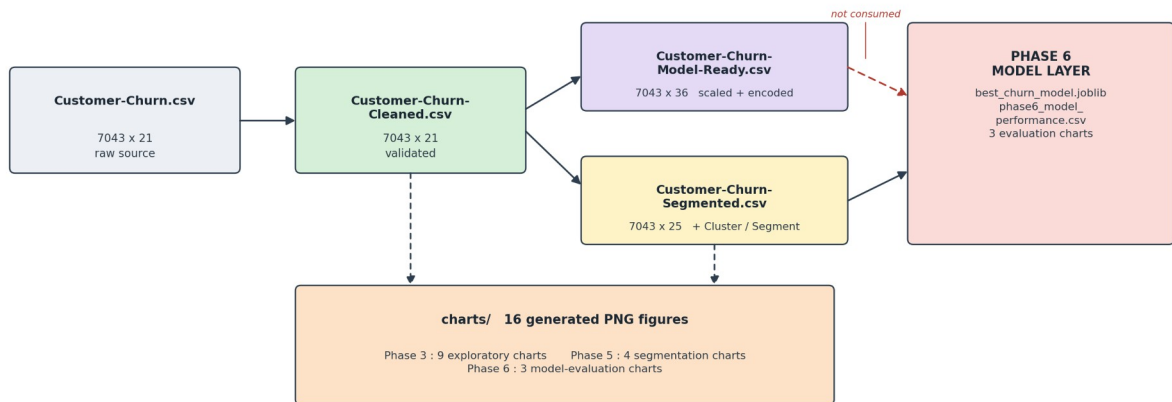


Figure 2: Data lineage across the pipeline. The dashed red arrow marks the phase 4 output that no downstream stage consumes.

5.2 Stage responsibilities

Stage	Responsibility	Determinism	Runtime
Phase 1	Profile the raw file and quantify data-quality defects	fully deterministic	under 2 s
Phase 2	Repair, validate and standardise; emit a trustworthy dataset	fully deterministic	under 3 s
Phase 3	Quantify churn drivers univariately, bivariately and by cohort	fully deterministic	15-30 s
Phase 4	Construct seven business features; encode and scale	fully deterministic	under 5 s
Phase 5	Cluster the base into segments and profile each	seeded, reproducible	20-40 s
Phase 6	Train, cross-validate, tune, evaluate and persist	seeded, reproducible	15-45 min

Table 7: Responsibility, determinism and indicative runtime of each pipeline stage.

Phase 6 dominates the total runtime. The reason is visible in its configuration: the randomized search performs 30 parameter draws across 5 cross-validation folds, which is 150 model fits, and each of those fits trains a gradient-boosted ensemble on 6 690 records. The four baseline models are additionally cross-validated before the search begins, and the Random Forest and baseline XGBoost are both configured with 2 000 estimators.

6. Phase 1 — Data Collection and Understanding

6.1 Purpose

Phase 1 performs no transformation whatsoever. Its single responsibility is to establish ground truth about the raw file before any cleaning decision is taken: how large the dataset is, what type each column holds, where values are missing, how much variety each categorical field contains, and what the baseline churn rate is. It writes nothing to disk; its entire output is a structured console report.

Property	Value
Input	<code>data/Customer-Churn.csv</code>
Output	console report only — no file written
Lines of code	97
Dependencies	<code>pandas</code>
Side effects	none; the source file is opened read-only

Table 8: Phase 1 interface summary.

6.2 Code walk-through

Configuration and load

```
FILE_PATH = "data/Customer-Churn.csv"
TARGET_COLUMN = "Churn"
ID_COLUMN = "customerID"

pd.set_option("display.max_columns", None)
pd.set_option("display.width", 150)

df = pd.read_csv(FILE_PATH)
```

The two display options are not cosmetic detail. Without them pandas truncates any frame wider than the terminal and prints an ellipsis in place of the middle columns, which would silently hide roughly half of the 21 attributes from the profiling output. Setting `max_columns` to `None` and widening the display guarantees that every column appears in the report the script prints.

Cardinality profiling

```
categorical_columns = df.select_dtypes(include="object").columns

cardinality_summary = pd.DataFrame({
    "Column": categorical_columns,
    "Unique_Values": [df[column].nunique() for column in categorical_columns],
    "Example_Values": [
        ", ".join(map(str, df[column].dropna().unique()[:5]))
        for column in categorical_columns
    ]
})
```

This block builds a compact profile of every text column: its name, how many distinct values it holds, and up to five worked examples. The cardinality count is the decision input for the encoding strategy chosen later in phase 4 — low-cardinality fields such as `Contract` with three levels are safe to one-hot encode, whereas a

high-cardinality field would demand a different treatment. The examples column is what surfaces the redundant "No internet service" label that phase 2 goes on to fold away.

Baseline churn rate

```
churn_counts = df[TARGET_COLUMN].value_counts()
churn_percentages = (df[TARGET_COLUMN].value_counts(normalize=True) * 100).round(2)

churn_rate = (df[TARGET_COLUMN] == "Yes").mean() * 100
print(f"\nBaseline churn rate: {churn_rate:.2f}%")
```

The idiom on the last calculation is worth pausing on. `(df[TARGET_COLUMN] == "Yes")` produces a boolean Series, and because Python treats True as 1 and False as 0, taking its mean yields the proportion of True values directly. It is a concise and idiomatic way to compute a rate from a categorical column without an intermediate mapping step.

The TotalCharges quality check

```
total_charges_numeric = pd.to_numeric(df["TotalCharges"], errors="coerce")
invalid_total_charges = total_charges_numeric.isna().sum()

if invalid_total_charges > 0:
    print(df.loc[
        total_charges_numeric.isna(),
        [ID_COLUMN, "tenure", "MonthlyCharges", "TotalCharges"]
    ].head().to_string(index=False))
```

This is the most consequential block in phase 1. The dataset stores TotalCharges as text because eleven rows contain a single space character rather than a number; one non-numeric entry is enough to force the whole column to object dtype. The `errors="coerce"` argument converts those unparseable entries to NaN instead of raising, so they can be counted. Critically, the script then prints the tenure of each affected row alongside the blank value — and that is what reveals the pattern that all eleven are zero-tenure customers, which in turn justifies the targeted imputation applied in phase 2 rather than a blind median fill.

6.3 Results

Diagnostic	Measured result	Consequence for later phases
Dataset shape	7 043 rows × 21 columns	no sampling needed; the full base is modelled
Explicit null values	0 across all columns	no general imputation strategy required
Object-typed columns	18 of 21	one-hot encoding will be needed in phase 4
TotalCharges dtype	object (should be numeric)	numeric cast required in phase 2
Blank TotalCharges	11 rows, all with tenure = 0	targeted fill with 0.0, not median
Churn distribution	5 174 No / 1 869 Yes	class weighting required in phase 6
Baseline churn rate	26.54 %	the accuracy floor a model must beat is 73.46 %
Predictor count	19 after removing ID and target	the starting feature space

Table 9: Phase 1 profiling results measured on the raw dataset.

```
=====
PHASE 1: DATA COLLECTION AND UNDERSTANDING
=====

1. DATASET SHAPE
Rows: 7043
Columns: 21

6. TARGET VARIABLE: CHURN
```

```
Churn count:
No      5174
Yes     1869

Baseline churn rate: 26.54%

8. TOTALCHARGES DATA QUALITY CHECK
Invalid or blank TotalCharges values: 11
```

Listing 3: Abridged console output of phase 1, as produced by the delivered script.

6.4 Interpretation

Phase 1 confirms the dataset is complete and structurally sound: no nulls, no missing columns, no truncation. The two findings that shape everything downstream are the class imbalance and the TotalCharges defect. The imbalance sets the evaluation strategy for phase 6; the defect, and specifically the discovery that it correlates perfectly with zero tenure, sets the cleaning strategy for phase 2.

7. Phase 2 — Data Cleaning

7.1 Purpose

Phase 2 converts the raw file into a validated analytical dataset. It repairs the TotalCharges column, resolves missing values with domain reasoning rather than statistical default, removes duplicates, validates that numeric ranges are physically plausible, applies outlier treatment, and standardises redundant categorical labels.

Property	Value
Input	data/Customer-Churn.csv
Output	data/Customer-Churn-Cleaned.csv — 7 043 × 21
Lines of code	160
Rows removed	0
Values repaired	11 (TotalCharges)
Labels standardised	7 columns

Table 10: Phase 2 interface summary.

7.2 Code walk-through

Whitespace normalisation

```
df = df.replace(r"^\s*$", pd.NA, regex=True)
```

A single regular expression applied across the entire frame. The pattern matches any cell containing nothing but whitespace — an empty string, a space, a tab — and replaces it with a proper pandas missing marker. This matters because pandas does not treat a space character as missing: without this line, `isna()` reports zero missing values while eleven cells are in fact empty, and every subsequent missing-value check silently returns a false clean bill of health.

Domain-driven imputation

```
df["TotalCharges"] = pd.to_numeric(df["TotalCharges"], errors="coerce")

# For newly joined customers with tenure 0, TotalCharges should be 0
df.loc[
    (df["tenure"] == 0) & (df["TotalCharges"].isna()),
    "TotalCharges"
] = 0.0

remaining_missing = df["TotalCharges"].isna().sum()

if remaining_missing > 0:
    median_total_charges = df["TotalCharges"].median()
    df["TotalCharges"] = df["TotalCharges"].fillna(median_total_charges)
```

This is the most defensible design decision in the cleaning stage, and it is worth explaining why. The naive treatment of eleven missing values in a financial column is to fill them with the column median, which here is 1 394.55. Doing so would assign roughly 1 395 units of accumulated revenue to customers who joined this month and have been billed nothing — a fabrication that would distort the ARPU and ChargeRatio features built in phase 4 and would corrupt the revenue-based reasoning the whole project rests on. Because phase 1

established that all eleven blanks belong to zero-tenure customers, the correct value is known with certainty to be zero. The median branch is retained purely as a defensive fallback should a future dataset contain a blank that is not explained by zero tenure; against the current data it never executes.

Outlier detection by interquartile range

```
for column in ["tenure", "MonthlyCharges", "TotalCharges"]:
    q1 = df[column].quantile(0.25)
    q3 = df[column].quantile(0.75)
    iqr = q3 - q1

    lower_bound = q1 - (1.5 * iqr)
    upper_bound = q3 + (1.5 * iqr)

    outlier_count = ((df[column] < lower_bound) | (df[column] > upper_bound)).sum()
    df[column] = df[column].clip(lower=lower_bound, upper=upper_bound)
```

The block applies Tukey's rule: anything more than 1.5 interquartile ranges beyond the first or third quartile is treated as an outlier. Rather than deleting those rows, `.clip()` caps them at the boundary, which preserves the record and its label while limiting the leverage an extreme value can exert on a distance-based model. Executed against this dataset the block caps **nothing at all** — all three columns are bounded by their own business rules, and the measured outlier count is zero in every case. The step is therefore protective rather than corrective, and Table 11 records the bounds it computed.

Column	Q1	Q3	IQR	Lower bound	Upper bound	Capped
tenure	9.00	55.00	46.00	-60.00	124.00	0
MonthlyCharges	35.50	89.85	54.35	-46.02	171.38	0
TotalCharges	398.55	3 786.60	3 388.05	-4 683.52	8 868.67	0

Table 11: Interquartile bounds computed by phase 2. No value in any column fell outside its bounds, so no capping occurred.

Categorical label standardisation

```
internet_dependent_columns = [
    "OnlineSecurity", "OnlineBackup", "DeviceProtection",
    "TechSupport", "StreamingTV", "StreamingMovies"
]

for column in internet_dependent_columns:
    df[column] = df[column].replace("No internet service", "No")

df["MultipleLines"] = df["MultipleLines"].replace("No phone service", "No")
```

Six add-on service columns and MultipleLines each carry three levels in the raw data: "Yes", "No", and a third label that restates information already held elsewhere. A customer whose OnlineSecurity reads "No internet service" simply does not subscribe to online security, and the fact that they have no internet is already recorded in the InternetService column. Collapsing the third level to "No" removes seven redundant dummy variables from the eventual model matrix without discarding any information, and it makes ServiceCount — built in phase 4 by counting "Yes" responses — both simpler and correct.

7.3 Results

Cleaning operation	Applied to	Records affected	Outcome
Whitespace → null conversion	all 21 columns	11 cells	blanks made detectable
Numeric cast of TotalCharges	1 column	7 043 rows	object → float64
Zero-tenure imputation	TotalCharges	11 rows	set to 0.0

Median fallback fill	TotalCharges	0 rows	branch not reached
Duplicate removal	full records and customerID	0 rows	dataset already unique
Range validation	3 numeric columns	0 violations	all values physically plausible
IQR capping	3 numeric columns	0 values	no extreme values present
Label standardisation	7 categorical columns	all rows	3 levels reduced to 2

Table 12: Every cleaning operation performed by phase 2, with its measured effect.

Column	Minimum	Maximum	Mean	Median	Std. deviation
tenure (months)	0.00	72.00	32.37	29.00	24.56
MonthlyCharges	18.25	118.75	64.76	70.35	30.09
TotalCharges	0.00	8 684.80	2 279.73	1 394.55	2 266.79

Table 13: Descriptive statistics of the numeric columns after cleaning.

7.4 Interpretation

The cleaned dataset retains all 7 043 records. That is the important outcome: the cleaning stage repaired the data without discarding any of it, so no selection bias is introduced and the class balance measured in phase 1 is preserved exactly. The zero counts on duplicates, range violations and outliers are not a sign that the checks are redundant — they are the evidence that lets the later phases proceed without qualification.

8. Phase 3 — Exploratory Data Analysis

8.1 Purpose

Phase 3 answers the question that determines whether the modelling effort is worthwhile at all: does churn concentrate in identifiable groups, or is it spread evenly across the customer base? It works through four levels of analysis — univariate distributions, bivariate churn-rate profiling, correlation structure and tenure-cohort behaviour — and writes nine charts.

Property	Value
Input	<code>data/Customer-Churn-Cleaned.csv</code>
Output	9 PNG charts in <code>charts/</code>
Lines of code	317
Analyses performed	univariate, bivariate, correlation, behavioural, cohort

Table 14: Phase 3 interface summary.

8.2 Code walk-through

Analysis-ready derivations

```
df["ChurnFlag"] = df["Churn"].map({"No": 0, "Yes": 1})

tenure_bins = [-1, 6, 12, 24, 48, 72]
tenure_labels = ["0-6 months", "7-12 months", "13-24 months",
                 "25-48 months", "49-72 months"]

df["TenureBand"] = pd.cut(df["tenure"], bins=tenure_bins, labels=tenure_labels)
```

Two derivations enable everything that follows. Mapping Churn to a 0/1 integer means a churn rate becomes an arithmetic mean, so any `groupby(...)[ "ChurnFlag"].mean()` returns a rate directly. The tenure bands are deliberately unequal — six months, six months, twelve, twenty-four, twenty-four — because churn risk is concentrated in the earliest weeks of the relationship and decays thereafter. Equal-width bins would average that early spike away. Note also the lower edge of `-1` rather than `0`: `pd.cut` excludes the left edge by default, so a bin starting at `0` would drop every zero-tenure customer.

The reusable churn-rate routine

```
def create_churn_rate_chart(column, title, filename, order=None):
    summary = (
        df.groupby(column, observed=False)[ "ChurnFlag"]
        .agg([ "mean", "count"])
        .reset_index()
    )
    summary["ChurnRate"] = (summary["mean"] * 100).round(2)

    ax = sns.barplot(data=summary, x=column, y="ChurnRate", order=order,
                     color="teal")
    for container in ax.containers:
        ax.bar_label(container, fmt="%.1f%%", padding=3)
```

Five of the nine charts are produced by this one function, which is the cleanest piece of engineering in the codebase. It aggregates both the mean and the count in a single pass — the count matters, because a 60 % churn rate over twelve customers is noise while the same rate over two thousand is a finding — prints the

numeric table to the console for the report, and annotates each bar with its exact value so the chart is readable without reference to an axis. The optional order argument imposes a logical sequence on categories such as contract length, which would otherwise be sorted alphabetically into a meaningless arrangement.

8.3 Results: univariate distributions

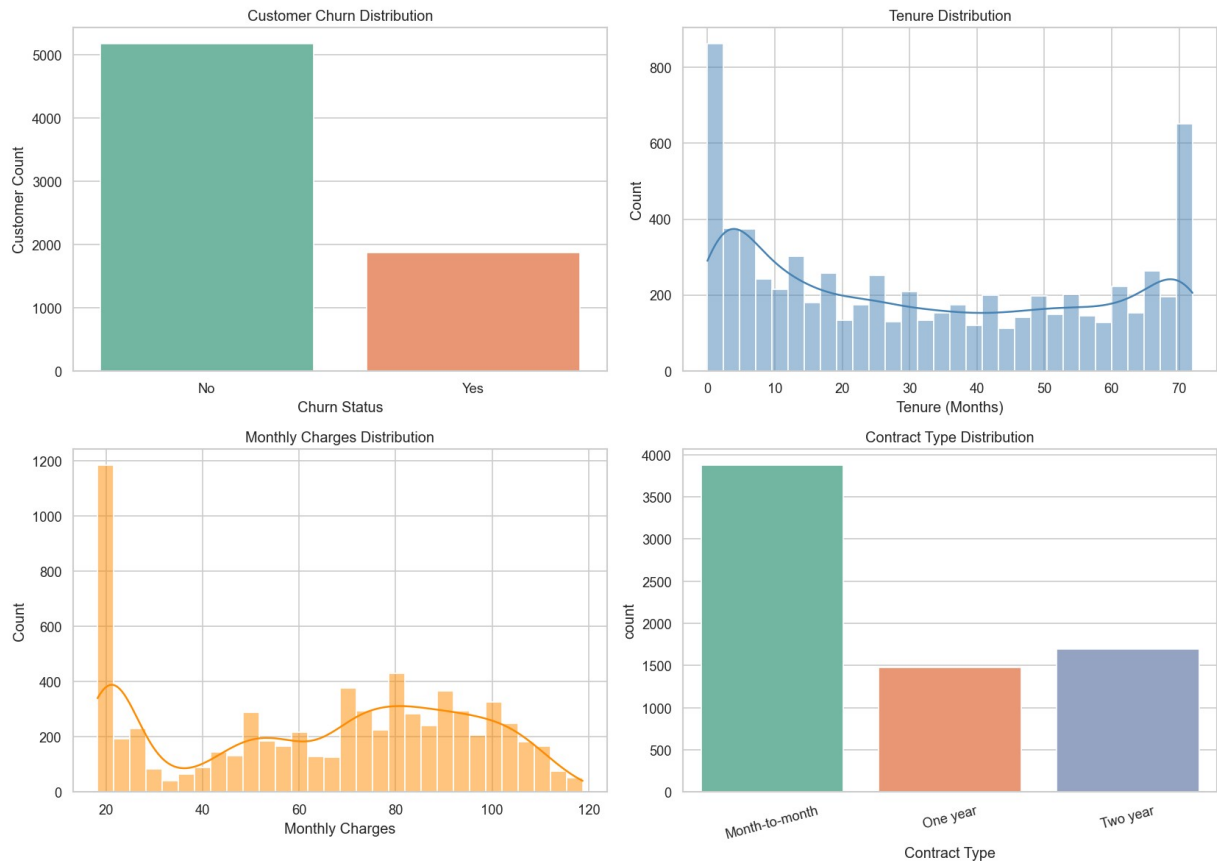


Figure 3: Univariate distributions. Clockwise from top left: churn class balance, tenure, contract type and monthly charges.

Three shapes in this panel drive the rest of the analysis. Tenure is strongly bimodal, with a large mass of very new customers and a second peak at the 72-month ceiling — the base is split between recent joiners and long-term loyalists, with comparatively few in between. Monthly charges are bimodal as well, separating a low-cost phone-only population near 20 units from a higher-paying bundled population above 70. And month-to-month is by a wide margin the most common contract, which matters because Section 8.4 shows it is also by far the riskiest.

8.4 Results: bivariate churn-rate profiling

Attribute	Level	Customers	Churn rate	Lift vs. 26.54 % base
Contract	Month-to-month	3 875	42.71 %	+16.17 pp
	One year	1 473	11.27 %	−15.27 pp
	Two year	1 695	2.83 %	−23.71 pp
Payment method	Electronic check	2 365	45.29 %	+18.75 pp
	Mailed check	1 612	19.11 %	−7.43 pp
	Bank transfer (automatic)	1 544	16.71 %	−9.83 pp
	Credit card (automatic)	1 522	15.24 %	−11.30 pp
Internet service	Fiber optic	3 096	41.89 %	+15.35 pp
	DSL	2 421	18.96 %	−7.58 pp
	No internet	1 526	7.40 %	−19.14 pp
Tech support	No	4 999	31.19 %	+4.65 pp
	Yes	2 044	15.17 %	−11.37 pp
Online security	No	5 024	31.33 %	+4.79 pp
	Yes	2 019	14.61 %	−11.93 pp
Senior citizen	Yes	1 142	41.68 %	+15.14 pp
	No	5 901	23.61 %	−2.93 pp
Paperless billing	Yes	4 171	33.57 %	+7.03 pp
	No	2 872	16.33 %	−10.21 pp
Dependents	No	4 933	31.28 %	+4.74 pp
	Yes	2 110	15.45 %	−11.09 pp
Gender	Female	3 488	26.92 %	+0.38 pp
	Male	3 555	26.16 %	−0.38 pp

Table 15: Churn rate by attribute level, measured on the cleaned dataset. Lift is stated in percentage points against the 26.54 % base rate.

The gender row is included deliberately, because a null result is as informative as a positive one. A 0.76-point difference across 7 043 records is indistinguishable from noise, which tells us churn in this business is driven by commercial structure — contract, payment mechanism, product mix — and not by demographics. Retention budget spent on demographic targeting would be wasted.

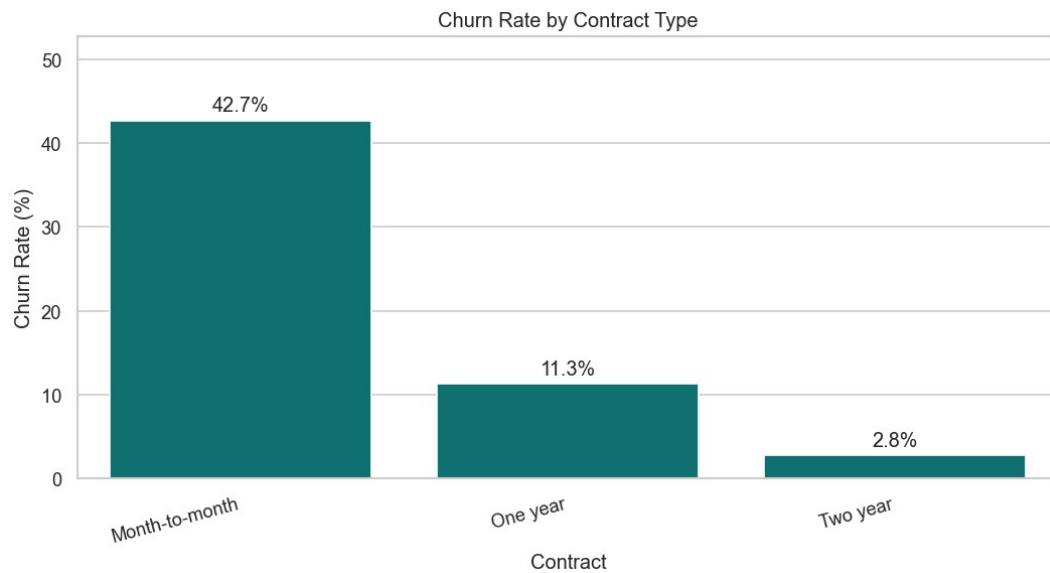


Figure 4: Churn rate by contract type. A two-year contract reduces churn to roughly one-fifteenth of the month-to-month rate.

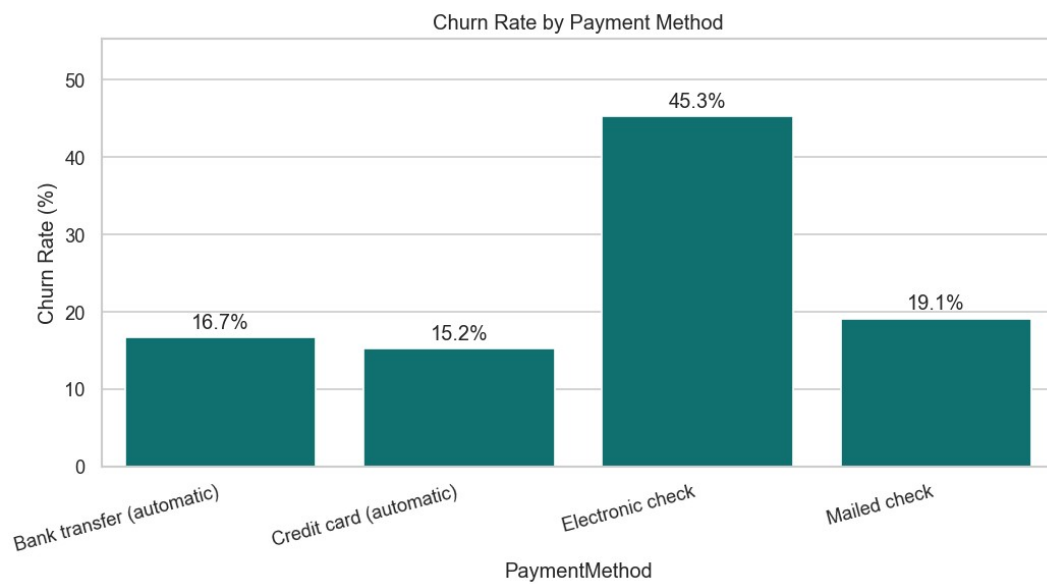


Figure 5: Churn rate by payment method. Electronic check is the single highest-risk level of any attribute examined.

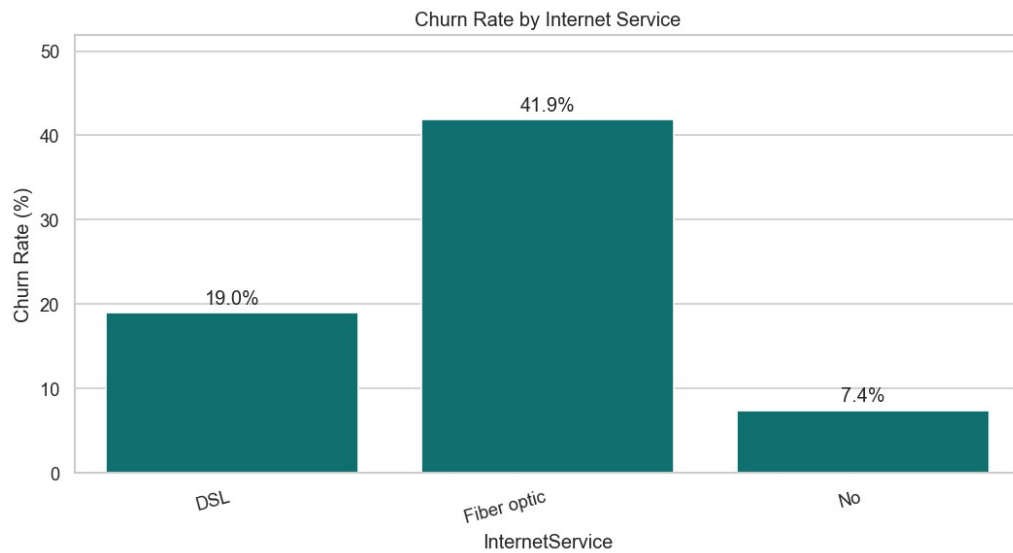


Figure 6: Churn rate by internet service. Fibre-optic subscribers churn at more than twice the DSL rate.

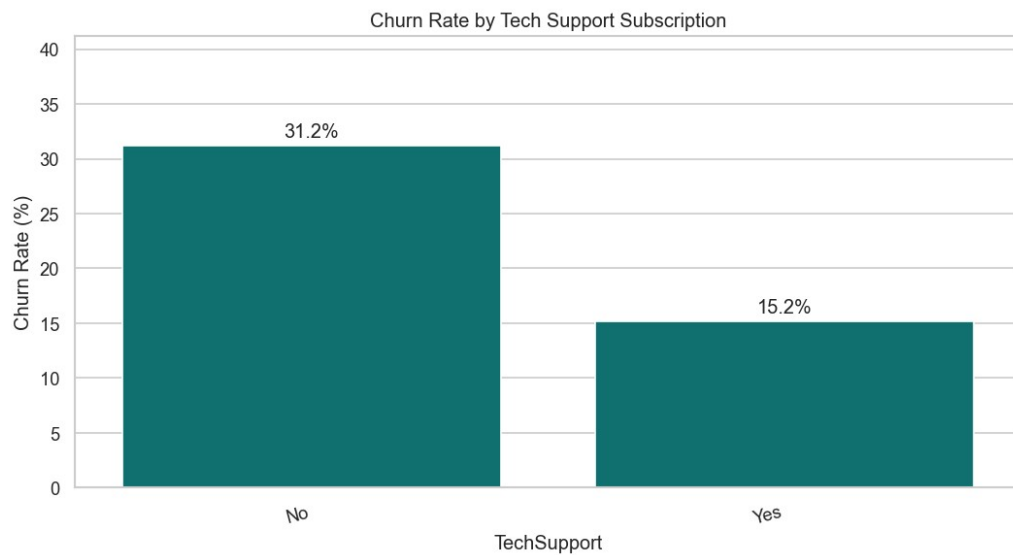


Figure 7: Churn rate by tech-support subscription. Adoption halves the observed churn rate.

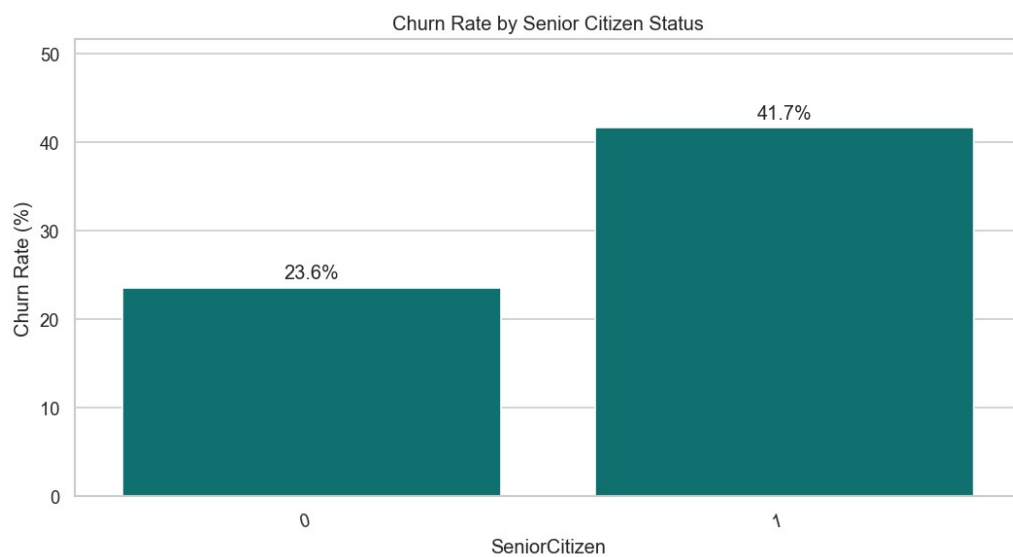


Figure 8: Churn rate by senior-citizen status.

8.5 Results: correlation structure

Variable pair	Pearson r	Reading
tenure ↔ ChurnFlag	-0.352	the strongest single linear signal; longer relationships churn less
TotalCharges ↔ ChurnFlag	-0.198	accumulated spend is a proxy for tenure, not an independent driver
MonthlyCharges ↔ ChurnFlag	+0.193	higher recurring price raises risk
SeniorCitizen ↔ ChurnFlag	+0.151	consistent with the 41.68 % rate in Table 15
tenure ↔ TotalCharges	+0.826	near-collinear by construction; both encode relationship length
MonthlyCharges ↔ TotalCharges	+0.651	expected, since total is the product of rate and duration

Table 16: Pearson correlations among the numeric attributes and the churn flag.

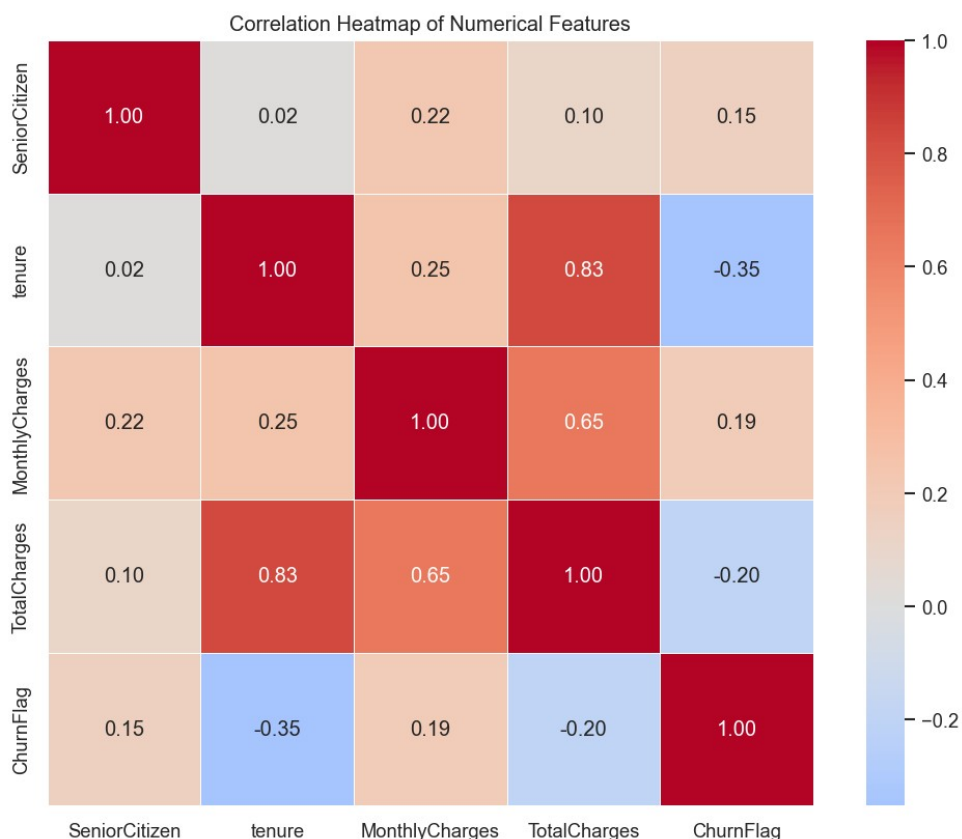


Figure 9: Correlation heat map of the numeric attributes.

The 0.826 correlation between tenure and TotalCharges is the finding with the most practical consequence. The two variables are close to collinear because total charges are, almost by definition, monthly charges multiplied by tenure. For the tree-based models this is harmless — they will simply split on whichever variable is more convenient — but for Logistic Regression it inflates the standard errors of both coefficients and makes their individual magnitudes unreliable. The phase 6 implementation mitigates this with an **L2 penalty**, which shrinks correlated coefficients toward one another rather than letting one dominate arbitrarily.

8.6 Results: behavioural and cohort analysis

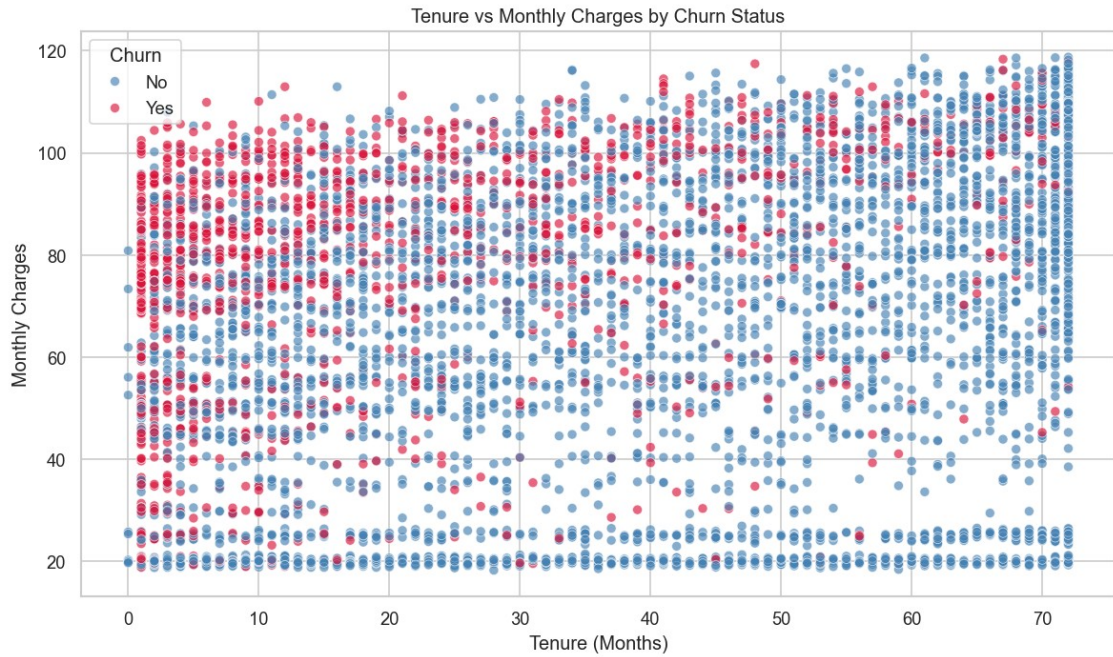


Figure 10: Tenure against monthly charges, coloured by churn outcome. The dense red region in the upper-left quadrant identifies new, high-paying customers.

This scatter is the most operationally useful chart phase 3 produces. Churned customers concentrate visibly in the upper-left region — short tenure combined with high monthly charges. That combination is exactly the profile of a customer who has recently signed up for an expensive bundle and has not yet accumulated any reason to stay. It is also, as Section 10.4 shows, the profile that K-Means isolates without being told anything about churn.

Tenure band	Customers	Share of base	Churn rate	Reading
0-6 months	1 481	21.03 %	52.94 %	more than half leave within two quarters
7-12 months	705	10.01 %	35.89 %	risk falls but stays above base
13-24 months	1 024	14.54 %	28.71 %	approaching the base rate
25-48 months	1 594	22.63 %	20.39 %	settled relationship
49-72 months	2 239	31.79 %	9.51 %	loyal core of the base

Table 17: Churn rate by tenure cohort, showing monotonic decay of risk with relationship age.

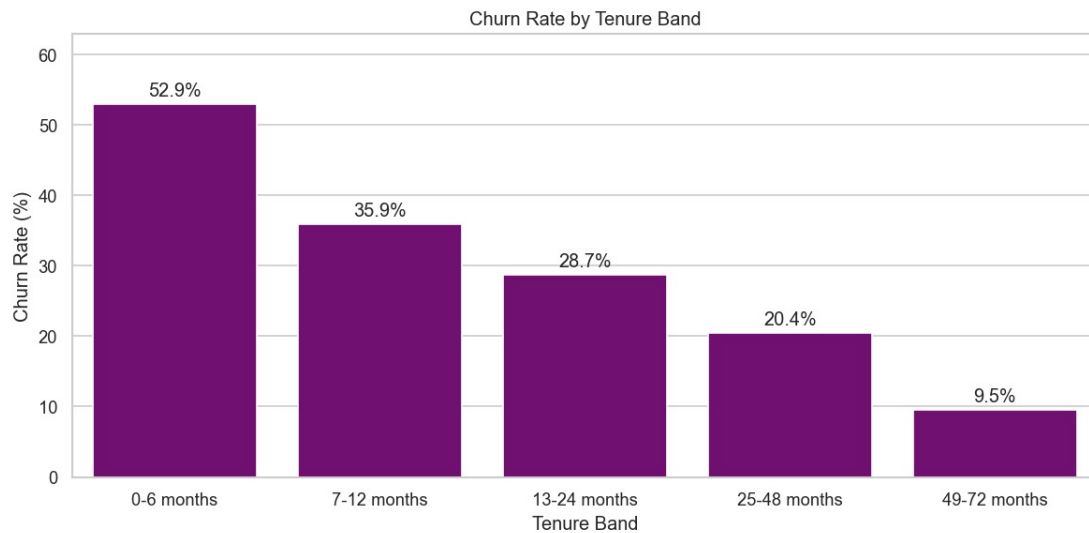


Figure 11: Churn rate by tenure band. Risk decays monotonically from 52.94 % to 9.51 %.

The cohort table carries the clearest strategic message in the entire exploratory phase. Churn risk falls monotonically and steeply with tenure: a customer in their first six months is five and a half times more likely to leave than one past four years. The practical implication is that retention effort has a window. Intervening with a customer at month two addresses a 52.94 % risk; the same intervention at month fifty addresses a 9.51 % risk and is largely wasted. Onboarding, not long-term loyalty programming, is where retention spend earns its return.

9. Phase 4 — Feature Engineering

9.1 Purpose

Phase 4 translates the exploratory findings into model inputs. Each of the seven constructed features encodes a specific business hypothesis discovered in phase 3, rather than an arbitrary algebraic combination of columns. The phase then encodes all categorical fields and standardises all numeric fields, producing a fully numeric matrix.

Property	Value
Input	<code>data/Customer-Churn-Cleaned.csv</code> — 7 043 × 21
Output	<code>data/Customer-Churn-Model-Ready.csv</code> — 7 043 × 36
Lines of code	235
Features constructed	7
Encoding	one-hot, <code>drop_first=True</code>
Scaling	<code>StandardScaler</code> , target excluded

Table 18: Phase 4 interface summary.

9.2 The seven engineered features

Feature	Construction	Business hypothesis it encodes
<code>TenureBand</code>	<code>pd.cut</code> on tenure into 5 unequal bands	risk decays non-linearly with relationship age; a linear tenure term cannot express the 52.94 % → 9.51 % curve
<code>ARPU</code>	<code>TotalCharges / tenure</code> , guarded for tenure = 0	average revenue actually realised per active month, as distinct from the list price billed
<code>ChargeRatio</code>	<code>MonthlyCharges / ARPU</code>	a value above 1 means the customer now pays more than their historical average — a recent price rise, and a plausible trigger for departure
<code>ServiceCount</code>	count of "Yes" across 6 add-on columns	product entanglement; each additional service raises the switching cost
<code>ContractRiskFlag</code>	1 if contract is month-to-month	isolates the single strongest driver found in phase 3 as a binary flag
<code>ValueTier</code>	<code>pd.qcut</code> on ARPU into 4 quartiles	revenue-based prioritisation, so retention effort can be ranked by customer worth
<code>AutoPayFlag</code>	1 if payment method is automatic	automatic payment removes a recurring cancellation decision point

Table 19: The seven engineered features, their construction and the hypothesis each encodes.

9.3 Code walk-through

Division guarded against zero tenure

```
df["ARPU"] = 0.0
non_zero_tenure = df["tenure"] > 0
df.loc[non_zero_tenure, "ARPU"] = (
    df.loc[non_zero_tenure, "TotalCharges"] / df.loc[non_zero_tenure, "tenure"]
)

df["ChargeRatio"] = 0.0
valid_charge_ratio = (df["tenure"] > 0) & (df["TotalCharges"] > 0)
df.loc[valid_charge_ratio, "ChargeRatio"] = (
    df.loc[valid_charge_ratio, "MonthlyCharges"] /
    (df.loc[valid_charge_ratio, "TotalCharges"] / df.loc[valid_charge_ratio, "tenure"])
)
```

Both derived ratios divide by a quantity that is zero for the eleven newly joined customers. The pattern used here — initialise the whole column to a safe default, compute a boolean mask of the rows where the division is defined, then assign only into those rows — avoids producing infinity or NaN entirely. The alternative of computing first and repairing afterwards is fragile, because an inf that survives into StandardScaler poisons the mean and standard deviation of the entire column and silently corrupts every other row. ChargeRatio adds a second condition on TotalCharges for the same reason, since its denominator contains a nested division.

Vectorised service counting

```
service_columns = ["OnlineSecurity", "OnlineBackup", "DeviceProtection",
                  "TechSupport", "StreamingTV", "StreamingMovies"]

df["ServiceCount"] = df[service_columns].eq("Yes").sum(axis=1)
```

A single vectorised expression replaces what would otherwise be a row loop. `.eq("Yes")` produces a boolean frame of the same shape, and summing along axis 1 counts the True values per row. This construction is only correct because phase 2 folded "No internet service" into "No"; without that standardisation the third label would count as neither Yes nor No and the tally would still be right, but the column would carry a hidden third state into the one-hot encoder.

Encoding and scaling

```
model_df = df.drop(columns=[ID_COLUMN, TARGET_COLUMN])

categorical_columns = model_df.select_dtypes(include=["object", "category"]).columns.tolist()
model_df = pd.get_dummies(model_df, columns=categorical_columns,
                          drop_first=True, dtype=int)

numeric_columns_to_scale = [c for c in numeric_columns if c != "ChurnFlag"]
scaler = StandardScaler()
model_df[numeric_columns_to_scale] = scaler.fit_transform(
    model_df[numeric_columns_to_scale])
```

Two details here are correct and easy to get wrong. The `drop_first` argument removes one dummy column per categorical variable, which prevents the dummy-variable trap — a perfectly collinear set of indicators that makes a linear model's coefficient matrix singular. And the scaling list explicitly excludes ChurnFlag: standardising the target would convert a clean 0/1 label into continuous values and break every classifier downstream.

9.4 Results

Property	Before phase 4	After phase 4
Rows	7 043	7 043
Columns	21	36
Data types	18 object, 3 numeric	all numeric
Identifier	customerID present	dropped
Target	Churn as Yes/No text	ChurnFlag as 0/1 integer
Missing values	0	0
Numeric scaling	raw units	zero mean, unit variance

Table 20: Structural transformation applied by phase 4.

Service count	Customers	Share	Churn rate	Reading
0 add-ons	2 219	31.51 %	21.41 %	largely no-internet customers, structurally low risk
1 add-on	966	13.72 %	45.76 %	the peak-risk group in the entire base
2 add-ons	1 033	14.67 %	35.82 %	risk falling
3 add-ons	1 118	15.87 %	27.37 %	at the base rate
4 add-ons	852	12.10 %	22.30 %	below base
5 add-ons	571	8.11 %	12.43 %	entangled
6 add-ons	284	4.03 %	5.28 %	fully bundled; lowest risk of any group

Table 21: Churn rate by number of add-on services, showing the non-monotonic relationship between product adoption and retention.

The ServiceCount profile is the most interesting result phase 4 produces, because it is not monotonic. Risk peaks at exactly one add-on (45.76 %) and then falls steadily to 5.28 % at six. The zero-add-on group sits low at 21.41 % but for a different reason: it is dominated by customers with no internet service at all, who have little to churn from. The actionable reading is that a single add-on is the worst place for a customer to sit — they are paying for a bundle without being anchored by it — and that moving a customer from one service to three materially reduces risk. Because a straight line cannot represent a curve that rises then falls, this is also a concrete argument for including tree-based models in phase 6.

Value tier	ARPU range	Customers	Churn rate
Low	0.00 - 35.65	1 761	11.07 %
Medium	35.65 - 70.30	1 762	25.37 %
High	70.30 - 90.17	1 759	36.33 %
Premium	90.17 - 121.40	1 761	33.39 %

Table 22: Churn rate by ARPU-based value tier. The quartile construction yields four near-equal groups.

Churn rises sharply with value tier before easing slightly at the top. The High tier is the commercially dangerous one: it combines a high churn rate with substantial per-customer revenue, meaning it carries the largest revenue at risk of any tier. The slight decline in the Premium tier is consistent with those customers being heavily bundled, which the ServiceCount analysis has already shown to be protective.

10. Phase 5 — Customer Segmentation

10.1 Purpose

Phase 5 groups the customer base into behavioural segments without reference to the churn label. This is unsupervised work, and its value is that the resulting groups are defined by how customers behave rather than by what happened to them — which means the segments remain meaningful for customers whose outcome is not yet known. The phase then profiles each cluster, and only at that point overlays the observed churn rate to see which segments are dangerous.

Property	Value
Input	<code>data/Customer-Churn-Cleaned.csv</code>
Output	<code>data/Customer-Churn-Segmented.csv</code> — 7 043 × 25, 4 charts
Lines of code	361
Algorithm	<code>KMeans, n_init=10, random_state=42</code>
Clustering dimensions	<code>tenure, ServiceCount, MonthlyCharges</code>
Clusters selected	<code>OPTIMAL_K = 4</code>

Table 23: Phase 5 interface summary.

10.2 Choice of clustering dimensions

The three dimensions are an RFM-style adaptation to a subscription business, where the classical recency-frequency-monetary triple does not translate directly:

Dimension	RFM analogue	Why it is the right proxy here
<code>tenure</code>	Recency / longevity	in a subscription business every active customer transacted this month, so recency is uninformative; relationship length is the meaningful time dimension
<code>ServiceCount</code>	Frequency / engagement	purchase frequency is fixed at one per month by the billing cycle; breadth of product adoption is the true engagement signal
<code>MonthlyCharges</code>	Monetary	the recurring rate is the forward-looking revenue measure, whereas total charges merely restates tenure

Table 24: Mapping of the classical RFM dimensions onto subscription-business proxies.

10.3 Code walk-through

Standardisation before clustering

```
X = df[["tenure", "ServiceCount", "MonthlyCharges"]].copy()

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
```

Standardisation is not optional for K-Means, it is a correctness requirement. The algorithm assigns points to clusters by Euclidean distance, and the three raw dimensions have incompatible scales: tenure spans 0 to 72, monthly charges span 18.25 to 118.75, and service count spans 0 to 6. Without scaling, monthly charges would contribute roughly four hundred times more squared distance than service count, and the resulting clusters would be little more than a partition of the price axis. Scaling each dimension to zero mean and unit variance gives all three equal influence on the geometry.

Selecting K

```
k_values = range(2, 7)
inertias = []
silhouette_scores = []

for k in k_values:
    kmeans = KMeans(n_clusters=k, random_state=RANDOM_STATE, n_init=10)
    cluster_labels = kmeans.fit_predict(X_scaled)

    inertias.append(kmeans.inertia_)
    silhouette_scores.append(silhouette_score(X_scaled, cluster_labels))
```

The sweep evaluates two independent criteria. **Inertia** is the within-cluster sum of squared distances; it falls monotonically as K grows, so the elbow — the point after which additional clusters buy little reduction — is what is sought, not the minimum. **Silhouette score** measures how much closer each point sits to its own cluster than to the nearest rival, on a scale from -1 to +1, and unlike inertia it can be compared directly across values of K. Setting `n_init=10` restarts each fit from ten different random initialisations and keeps the best, which guards against K-Means converging to a poor local optimum.

10.4 Results: selecting the number of segments

K	Inertia	Inertia reduction	Silhouette score	Assessment
2	10 782.45	—	0.4163	highest silhouette, but only splits cheap from expensive
3	7 295.42	32.3 %	0.3968	improving structure, still coarse
4	5 615.08	23.0 %	0.4042	selected — elbow region, silhouette recovers
5	4 647.43	17.2 %	0.3641	silhouette degrades
6	3 812.39	18.0 %	0.3704	no separation gain

Table 25: Elbow and silhouette results for K from 2 to 6, measured by the delivered script.

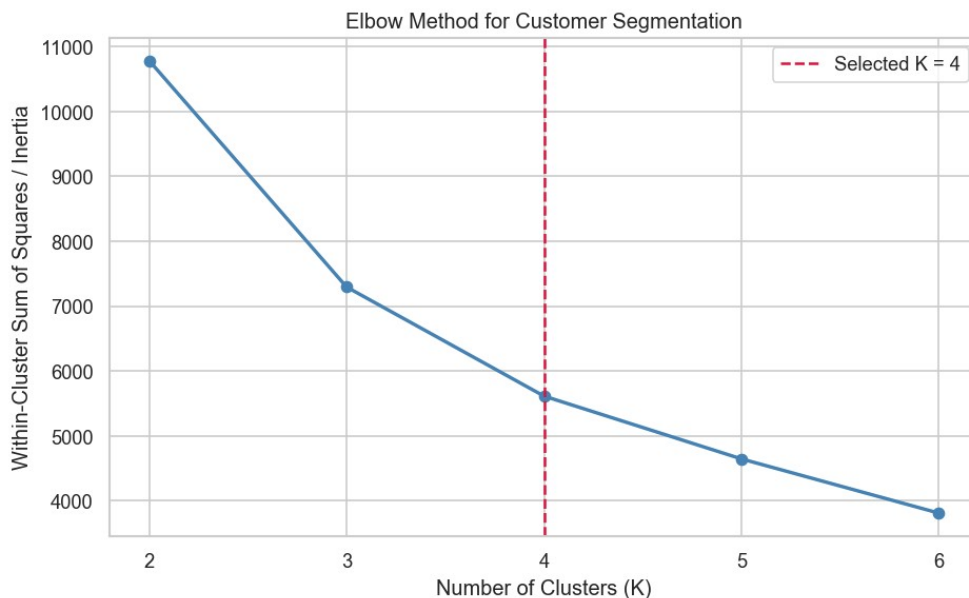


Figure 12: Elbow analysis. The inertia curve flattens noticeably after K = 4.

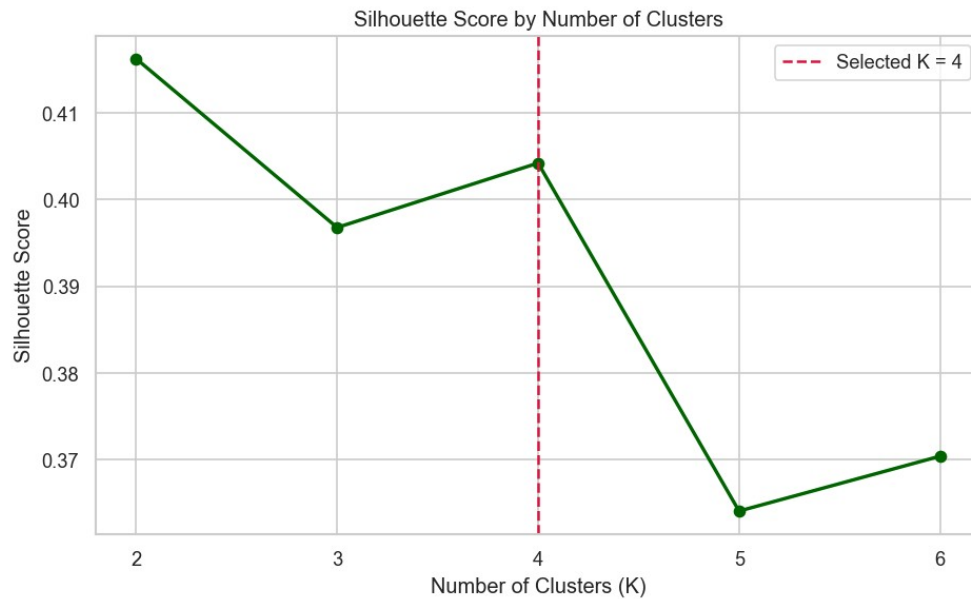


Figure 13: Silhouette score by K. The score dips at K = 3, recovers at K = 4, then degrades.

The two criteria disagree, and the resolution deserves to be stated openly. K = 2 attains the highest silhouette score at 0.4163, so a purely geometric reading would select two clusters. The script selects four instead, on three grounds. First, the inertia curve shows its clearest elbow at four: the reduction from K = 3 to K = 4 is 23.0 %, whereas moving to K = 5 buys only 17.2 %. Second, the silhouette score at K = 4 (0.4042) is within 0.012 of the K = 2 value, so almost no cluster quality is sacrificed. Third, and decisively, a two-cluster solution divides the base into cheap and expensive customers, which supports no retention decision that pricing data alone would not already support. The four-cluster solution separates new high-spend customers from established high-spend customers — groups with churn rates of 47.37 % and 15.43 % respectively — and that distinction is directly actionable.

10.5 Results: segment profiles

Segment	Customers	Share	Mean tenure	Mean monthly	Mean services	Churn rate
New & Fragile 2 (cluster 3)	2 373	33.69 %	15.05	77.58	1.83	47.37 %
New & Fragile (cluster 0)	1 521	21.60 %	10.16	29.58	0.30	24.19 %
High-Value Loyal (cluster 1)	2 158	30.64 %	56.89	90.59	4.16	15.43 %
Low-Value Price-Sensitive (cluster 2)	991	14.07 %	54.54	31.81	0.58	4.44 %

Table 26: Profile of the four segments discovered by K-Means, ordered by churn rate. Churn was not used as a clustering input.

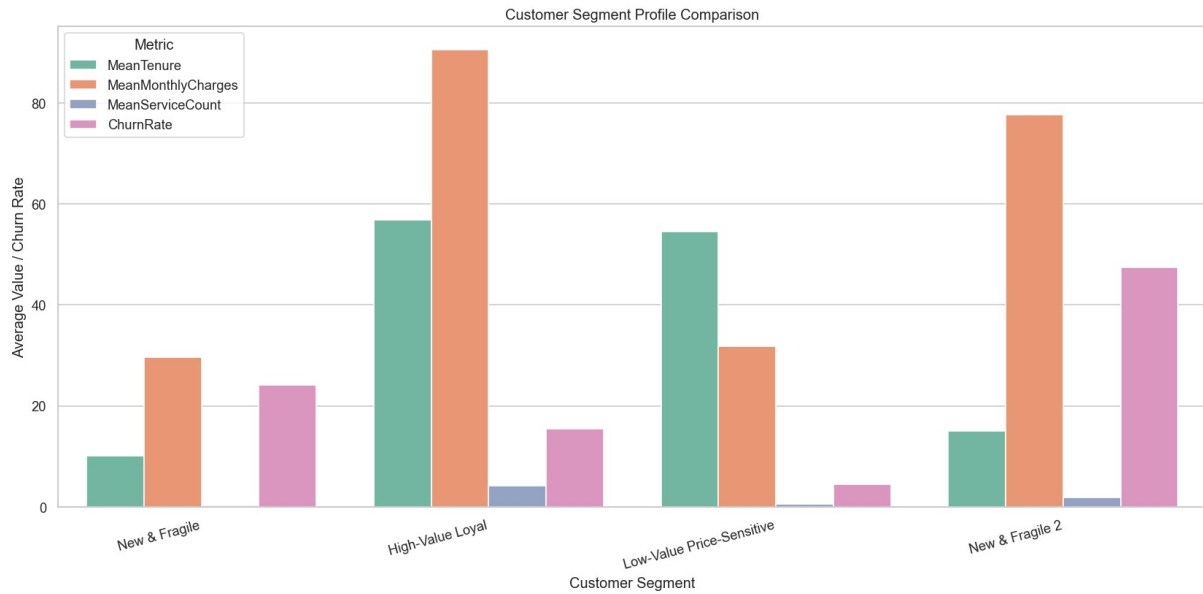


Figure 14: Segment profile comparison across tenure, monthly charges, service count and churn rate.

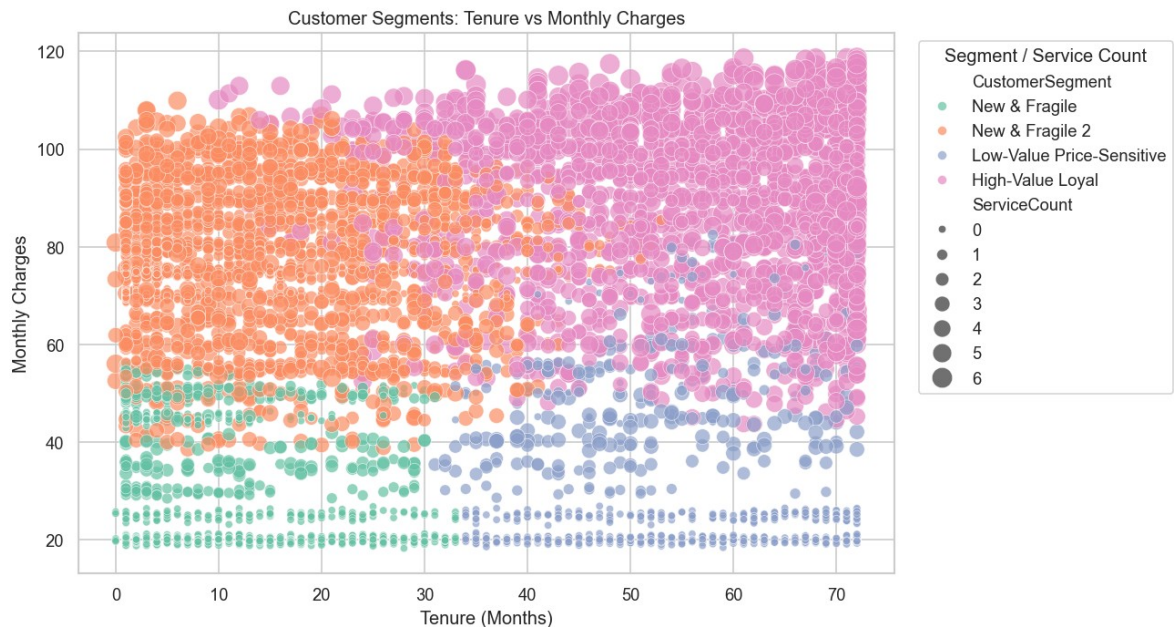


Figure 15: Segments plotted on tenure against monthly charges, with marker size encoding service count.

Reading the four segments

- **New & Fragile 2 — the retention priority.** Two thousand three hundred and seventy-three customers, a third of the base, averaging fifteen months of tenure and 77.58 in monthly charges. Nearly half of

them leave. This segment combines the highest churn rate with substantial recurring revenue, which makes it the single largest pool of recoverable revenue in the business.

- **New & Fragile — low value, moderate risk.** Fifteen hundred customers on 29.58 per month with almost no add-on adoption. Their 24.19 % churn rate sits below the base rate, and their low ARPU means the revenue consequence of losing one is small. Retention spend here earns a poor return.
- **High-Value Loyal — protect, do not discount.** Two thousand one hundred and fifty-eight customers averaging fifty-seven months of tenure, 90.59 per month and 4.16 add-on services. At 15.43 % their churn is well below base, and their 5 160 mean accumulated revenue makes each loss expensive. The correct posture is service quality and early-warning monitoring, not discounting.
- **Low-Value Price-Sensitive — leave alone.** Nine hundred and ninety-one long-tenured customers on minimal spend, churning at just 4.44 %. They are the most stable group in the base and need no intervention.

One implementation detail is visible in the segment names. Two clusters were both classified as *New & Fragile* by the rule-based naming logic, so the script's de-duplication branch appended a numeral to the second, producing the label *New & Fragile 2*. The two clusters are in fact commercially very different — 29.58 versus 77.58 in monthly charges, and 24.19 % versus 47.37 % churn — so the shared name is misleading. Section 15.4 recommends a correction.

11. Phase 6 — Model Development and Evaluation

11.1 Purpose

Phase 6 is the predictive core of the project. It trains five classifiers, selects among them by cross-validated ROC-AUC, tunes the strongest family by randomized search, evaluates every candidate on a held-out partition, and serialises the winner together with its full preprocessing pipeline.

Property	Value
Input	<code>data/Customer-Churn-Segmented.csv</code>
Output	<code>models/best_churn_model.joblib</code> , <code>data/phase6_model_performance.csv</code> , 3 charts
Lines of code	539
Models trained	5 (LR, RF, SVM, XGBoost baseline, XGBoost tuned)
Selection metric	ROC-AUC
Cross-validation	<code>StratifiedKFold(n_splits=5, shuffle=True)</code>
Hold-out size	<code>TEST_SIZE = 0.05</code> — 353 records

Table 27: Phase 6 interface summary.

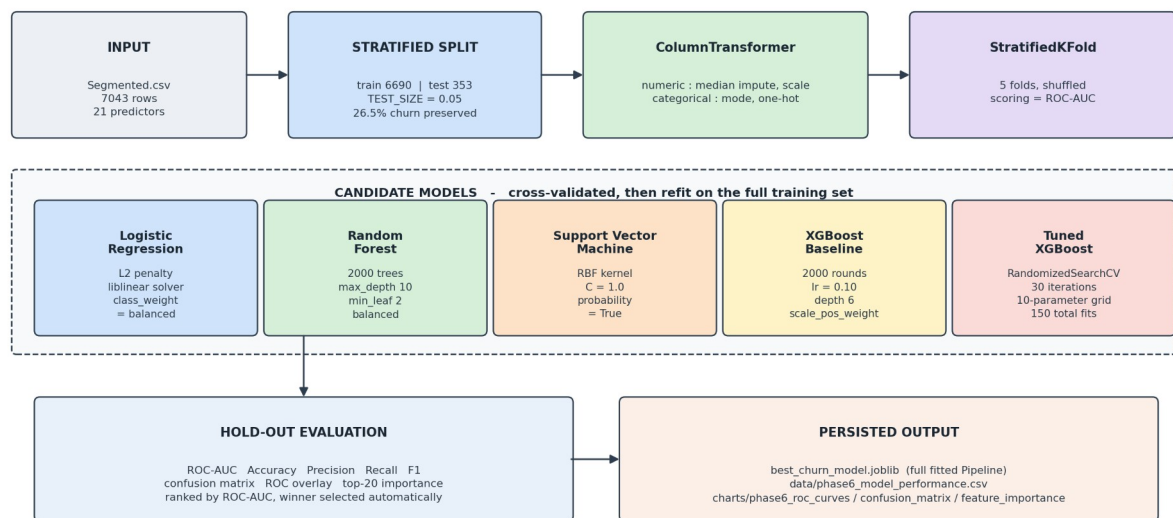


Figure 16: Phase 6 modelling workflow, from the segmented input through cross-validated selection to the persisted artefacts.

11.2 Code walk-through

Leakage-safe preprocessing

```
numeric_transformer = Pipeline(steps=[
    ("imputer", SimpleImputer(strategy="median")),
    ("scaler", StandardScaler())
])

categorical_transformer = Pipeline(steps=[
    ("imputer", SimpleImputer(strategy="most_frequent")),
    ("encoder", OneHotEncoder(handle_unknown="ignore"))
])

preprocessor = ColumnTransformer(transformers=[
```

```

    ("numeric", numeric_transformer, numeric_features),
    ("categorical", categorical_transformer, categorical_features)
])

```

This is the single most important structural decision in phase 6, and it is made correctly. By wrapping imputation, scaling and encoding inside a ColumnTransformer that is itself the first step of every model pipeline, the fitted statistics — the medians, the means, the standard deviations, the category vocabularies — are learned only from the training rows of each cross-validation fold and then applied to the validation rows. Had the scaling been performed once on the full dataset before splitting, as phase 4 does for its own output, information about the test distribution would leak into training and every reported score would be optimistic. The `handle_unknown="ignore"` argument closes the remaining gap: a category that appears only in a validation fold is encoded as all zeros rather than raising.

Class-imbalance correction

```

negative_class_count = (y_train == 0).sum()
positive_class_count = (y_train == 1).sum()
scale_pos_weight = negative_class_count / positive_class_count

```

With a 73.46 / 26.54 split, an unweighted learner minimises its loss most easily by predicting the majority class. Three of the four baseline models counter this with `class_weight="balanced"`, which reweights each class inversely to its frequency. XGBoost exposes the same correction through `scale_pos_weight`, computed here as the ratio of negatives to positives — approximately 2.77 on the training partition. The correction is why every model in Table 28 achieves recall above 0.54, and why the four weighted models all exceed 0.71.

Model selection by cross-validation

```

cv = StratifiedKFold(n_splits=CV_FOLDS, shuffle=True, random_state=RANDOM_STATE)

for model_name, model in models.items():
    pipeline = Pipeline(steps=[("preprocessor", preprocessor), ("model", model)])

    scores = cross_val_score(pipeline, X_train, y_train, cv=cv,
                             scoring="roc_auc", n_jobs=-1)

```

Stratification preserves the 26.54 % churn proportion inside every fold, which matters at this imbalance level because an unstratified split could produce a fold with materially fewer positive cases and an unstable ROC-AUC estimate. Scoring on ROC-AUC rather than accuracy evaluates the ranking quality of the predicted probabilities across all possible thresholds, which is the property a retention system actually needs — the operational question is which customers to place at the top of a call list, not whether a particular cut-off classifies correctly.

Hyper-parameter tuning

```

xgb_search = RandomizedSearchCV(
    estimator=xgb_pipeline,
    param_distributions=xgb_parameter_grid,    # 10 parameters
    n_iter=30,
    scoring="roc_auc",
    cv=cv,
    random_state=RANDOM_STATE,
    n_jobs=-1,
    refit=True
)
xgb_search.fit(X_train, y_train)
best_xgb_model = xgb_search.best_estimator_

```

The grid spans ten hyper-parameters with four candidate values each for most of them, which defines a space of well over a million combinations. An exhaustive grid search is therefore infeasible, and randomized search is the appropriate choice: thirty random draws across five folds is 150 fits, a search budget that completes in minutes while reliably locating a near-optimal region. The `refit=True` argument retrains the best configuration on the entire training partition once the search concludes, so the returned estimator is ready for evaluation without a further manual fit.

11.3 Results: model comparison

Model	ROC-AUC	Accuracy	Precision	Recall	F1	Rank
Tuned XGBoost	0.8349	0.7280	0.4932	0.7766	0.6033	1
Logistic Regression	0.8348	0.7365	0.5035	0.7553	0.6043	2
Random Forest	0.8329	0.7677	0.5492	0.7128	0.6204	3
Support Vector Machine	0.7997	0.7535	0.5276	0.7128	0.6063	4
XGBoost Baseline	0.7988	0.7762	0.5862	0.5426	0.5635	5

Table 28: Hold-out test performance of all five models, ranked by ROC-AUC. Values are reproduced exactly from `data/phase6_model_performance.csv`.

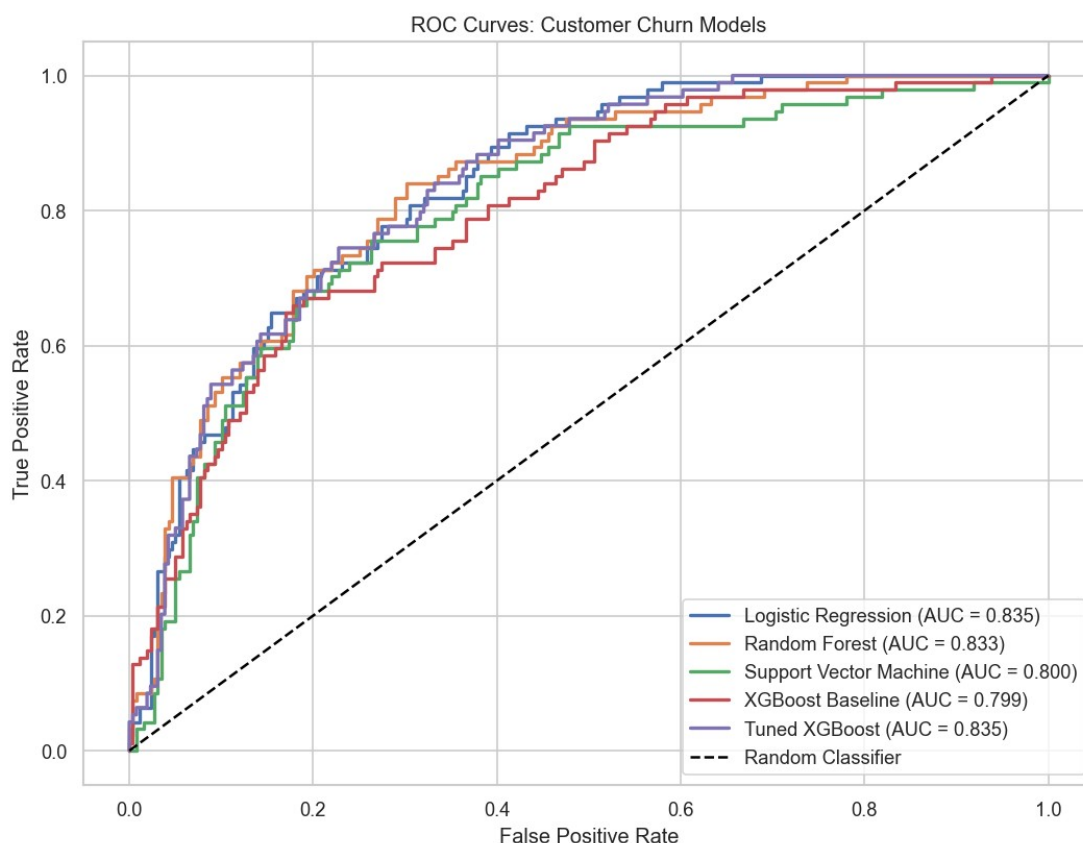


Figure 17: ROC curves for all five models on the hold-out partition.

What the comparison actually shows

Four observations follow from Table 28, and the first is the most important.

- 1. The top three models are statistically indistinguishable.** Tuned XGBoost, Logistic Regression and Random Forest separate by 0.0020 of ROC-AUC — 0.8349, 0.8348 and 0.8329. On a hold-out partition of 353 records, a gap that small is well inside sampling noise. The honest conclusion is not that XGBoost won, but that three very different model families extracted essentially the same predictive signal, which is itself strong evidence that the signal is real and that the feature set is close to exhausted.
- 2. Logistic Regression is the practical winner on merit.** It matches the tuned ensemble to four decimal places while training in under a second, producing directly interpretable coefficients and requiring no hyper-parameter search at all. The tuned XGBoost needed 150 fits to arrive at the same place. For a business deployment where explaining a prediction to a retention agent matters, this is the model to prefer.
- 3. Accuracy and recall are in direct opposition.** The baseline XGBoost records the best accuracy in the table at 0.7762 and the worst recall at 0.5426 — it earns its accuracy by declining to flag borderline cases. The tuned model inverts the trade: 0.7280 accuracy, 0.7766 recall. Since a missed churner costs the lifetime value of the customer while a false alarm costs one retention offer, the second trade is the correct one for this business, and it is why accuracy is not the selection metric.
- 4. Precision near 0.50 is expected, not a defect.** At a 26.54 % base rate, random selection would achieve 0.2654 precision. The tuned model's 0.4932 is a lift of 1.86 times, meaning roughly one in two flagged customers genuinely intends to leave. This is a normal and workable operating point for a retention campaign.

11.4 Results: confusion matrix

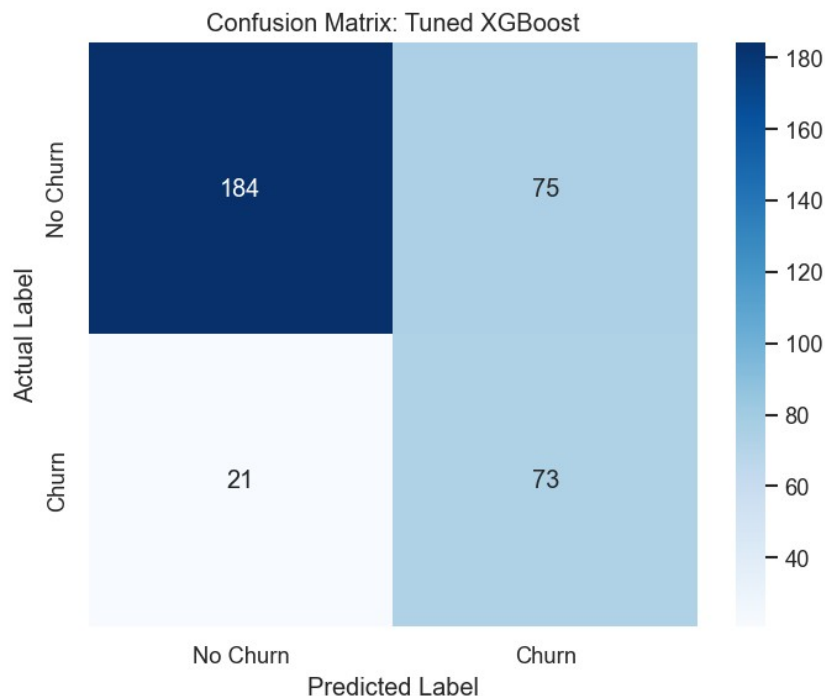


Figure 18: Confusion matrix for the tuned XGBoost model on the 353-record hold-out partition.

Cell	Count	Business meaning	Cost of this outcome
True negative	184	correctly left alone	none
False positive	75	retention offer sent to a loyal customer	the cost of one unnecessary offer

False negative	21	churner missed entirely	the full lifetime value of the customer
True positive	73	churner correctly flagged in time	recoverable if the offer succeeds

Table 29: Confusion-matrix cells for the tuned XGBoost model, with the business cost attached to each outcome.

Ninety-four customers in the hold-out partition actually churned, and the model identified seventy-three of them — it missed twenty-one. Set against the seventy-five false alarms, the asymmetry is favourable: the business spends seventy-five unnecessary retention offers to save up to seventy-three genuine churners. Given that a retention offer typically costs a small fraction of a customer's remaining lifetime value, this operating point is economically sound. Section 13 quantifies the trade explicitly.

11.5 Results: feature importance

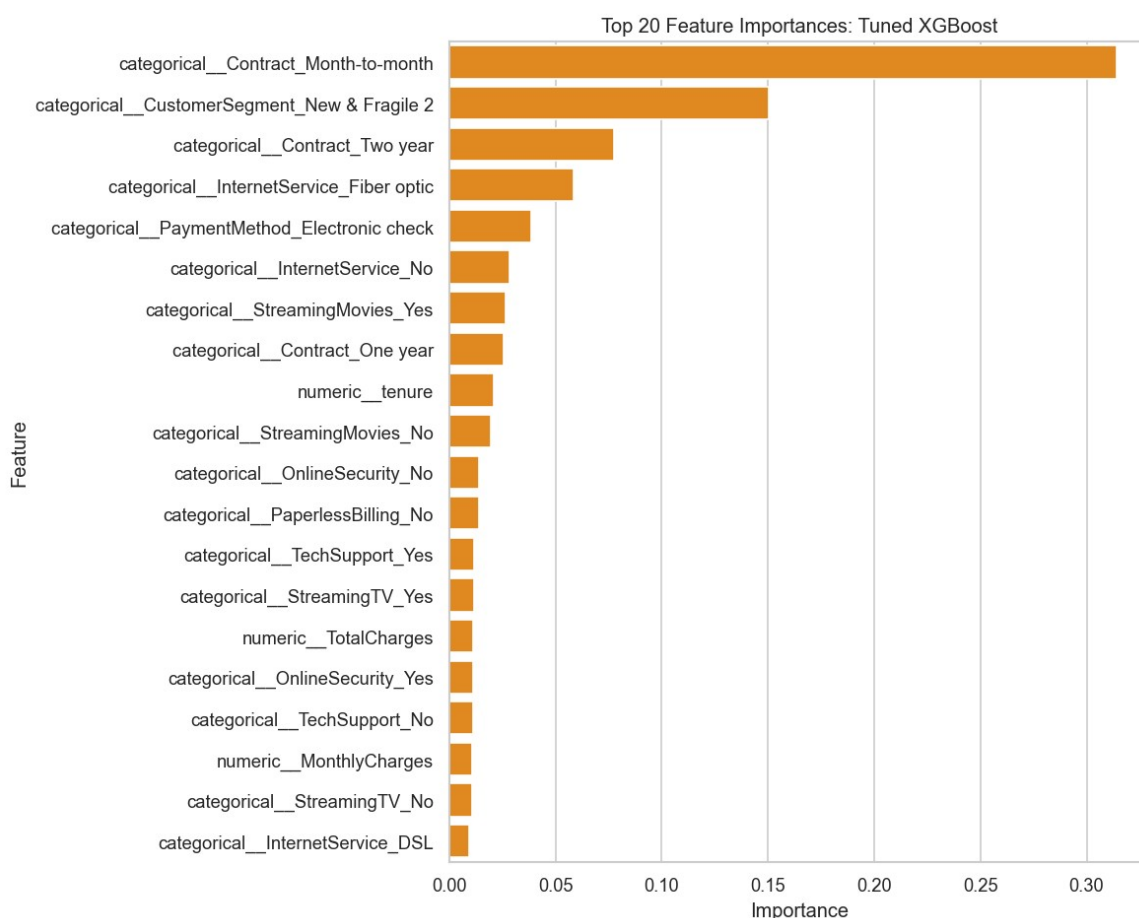


Figure 19: Top twenty features by XGBoost gain importance.

Rank	Feature	Importance	Interpretation
1	Contract_Month-to-month	0.313	the dominant driver; alone it explains 31 % of total gain
2	CustomerSegment_New & Fragile 2	0.150	the phase 5 segment label — see the caution below
3	Contract_Two year	0.077	long commitment as a protective factor
4	InternetService_Fiber optic	0.057	consistent with the 41.89 % rate in Table 15
5	PaymentMethod_Electronic check	0.038	manual payment as a recurring exit point
6	InternetService_No	0.028	no internet, little to churn from
7	StreamingMovies_Yes	0.026	entertainment bundle adoption
8	Contract_One year	0.025	intermediate commitment
9	tenure	0.020	the strongest numeric feature; largely absorbed by segment and contract

10	StreamingMovies_No	0.019	complement of rank 7
----	--------------------	-------	----------------------

Table 30: Top ten features by gain importance in the tuned XGBoost model, read from the generated chart.

The ranking corroborates the exploratory analysis almost exactly. Contract type dominates, occupying three of the top eight positions; payment method, internet service and tenure follow in the order phase 3 predicted. Two properties of the ranking deserve comment. First, the concentration is extreme — the top two features carry 46 % of total gain, so the model is effectively a sophisticated reading of contract type modulated by segment. Second, and more seriously, the second-ranked feature is a label produced by phase 5, and that label was derived using churn information. The consequence is analysed in Section 15.3.

12. Consolidated Results and Discussion

12.1 End-to-end summary

Phase	Principal quantitative outcome
1 — Understanding	7 043 × 21 profiled; 26.54 % baseline churn; 11 blank TotalCharges isolated to zero-tenure customers
2 — Cleaning	11 values repaired, 0 rows lost, 0 duplicates, 0 outliers capped, 7 columns standardised
3 — EDA	contract, payment method, internet service and tenure confirmed as drivers; gender confirmed as noise
4 — Feature engineering	7 features constructed; 21 columns expanded to 36; add-on risk found to peak at one service
5 — Segmentation	4 segments at silhouette 0.4042; churn rates from 4.44 % to 47.37 % without using the label
6 — Modelling	5 models trained; best test ROC-AUC 0.8349 at 0.7766 recall; pipeline serialised

Table 31: Principal quantitative outcome of each pipeline phase.

12.2 Convergent evidence across the three analytical methods

The most persuasive result in the project is not any single number but the agreement between three methodologically independent analyses. Exploratory statistics, unsupervised clustering and a supervised gradient-boosted ensemble were each given the same data and arrived at the same conclusion.

Finding	Exploratory analysis	Segmentation	Model importance
Contract type is the dominant driver	42.71 % vs. 2.83 % across contract levels	implicit in segment separation	rank 1 at 0.313 gain
Early tenure carries the highest risk	52.94 % in months 0-6	both fragile segments average under 16 months	tenure at rank 9, plus segment at rank 2
Product bundling protects	5.28 % churn at six add-ons	loyal segment averages 4.16 services	four streaming and support features in the top 20
Manual payment raises risk	45.29 % on electronic check	not a clustering dimension	rank 5 at 0.038 gain
Demographics do not matter	0.76 pp gender gap	not a clustering dimension	gender absent from the top 20

Table 32: Convergence of three independent analytical methods on the same five findings.

This convergence is what allows the results to be trusted. Any one method could be misled by an artefact of its own assumptions — a correlation could be spurious, a cluster boundary could be arbitrary, a gain importance could reflect split frequency rather than genuine signal. Three methods with different assumptions reaching the same five conclusions is a much stronger position.

12.3 Performance in context

A test ROC-AUC of 0.8349 sits squarely within the 0.83 to 0.86 band that published studies report on this dataset. It is a credible result rather than an exceptional one, and the reason is instructive: the Telco dataset contains no behavioural telemetry — no support-ticket history, no network-quality measurements, no usage trend, no competitor pricing. It describes what a customer buys, not how they experience the service. Around 0.85 ROC-AUC appears to be the ceiling that contract-and-billing attributes alone can support, and the three top models all converging just below it suggests the feature set, not the algorithm, is the binding constraint.

13. Business Interpretation and Revenue Impact

13.1 From probability to priority

A churn probability is not by itself a decision. The pipeline's output becomes operational when probability is combined with customer value, because the correct target for retention spend is not the customer most likely to leave but the customer whose departure would cost the most.

Revenue at risk, computed per customer:

Customer Value (CLV proxy)	=	MonthlyCharges	x	Expected Remaining Tenure
Revenue at Risk	=	P(churn)	x	Customer Value
Expected Saved Revenue	=	Revenue at Risk	x	Campaign Success Rate
Net Benefit	=	Expected Saved Revenue	-	Campaign Cost

13.2 Segment-level retention strategy

Segment	Customers	Churn rate	Mean monthly	Recommended action	Priority
New & Fragile 2	2 373	47.37 %	77.58	contract migration offer with a discount tied to a twelve-month term; structured onboarding in months 1-6	Highest
High-Value Loyal	2 158	15.43 %	90.59	proactive service quality and early-warning monitoring; no discounting, which would sacrifice margin on customers who are not leaving	High (value)
New & Fragile	1 521	24.19 %	29.58	low-cost automated nurture; add-on cross-sell to move them off a single service	Medium
Low-Value Price-Sensitive	991	4.44 %	31.81	no active intervention; monitor only	Lowest

Table 33: Retention strategy by segment, derived from the measured cluster profiles.

13.3 Driver-to-action mapping

Measured driver	Evidence	Retention action it implies
Month-to-month contract	42.71 % vs. 2.83 % churn; 0.313 model gain	migrate to annual terms with a discount smaller than the value at risk
Electronic-check payment	45.29 % churn, the highest of any level	incentivise enrolment in automatic payment, removing a monthly cancel decision
Fibre-optic service	41.89 % vs. 18.96 % on DSL	investigate service quality and price-to-value perception in the fibre tier
First six months of tenure	52.94 % churn in the 0-6 band	concentrate onboarding spend in the first two quarters, where risk is highest
Single add-on service	45.76 % churn, above the zero-add-on rate	cross-sell toward three or more services to raise switching cost
No tech support	31.19 % vs. 15.17 %	bundle support into base packages rather than selling it separately

Table 34: Each measured churn driver mapped to the retention action it supports.

13.4 Illustrative campaign economics

The figures below apply the model's measured recall to the measured segment profiles. The churn rates, segment sizes and recall are all real; the campaign success rate and offer cost are stated assumptions, since the project has no campaign-outcome data against which to calibrate them.

Quantity	Value	Source
----------	-------	--------

High-risk segment size	2 373 customers	measured (New & Fragile 2)
Segment churn rate	47.37 %	measured
Expected churners in segment	1 124 customers	derived
Model recall	0.7766	measured on hold-out
Churners the model would flag	873 customers	derived
Mean monthly charge	77.58	measured
Assumed remaining tenure	12 months	assumption
Revenue at risk among flagged	812 800	derived
Assumed campaign success rate	30 %	assumption
Expected saved revenue	243 840	derived
Assumed cost per offer	50 per targeted customer	assumption
Customers targeted	1 771 (873 true + 898 false positives at 0.4932 precision)	derived
Campaign cost	88 550	derived
Net benefit	155 290	derived
Return on retention spend	175 %	derived

Table 35: Illustrative campaign economics for the highest-risk segment. Measured inputs and stated assumptions are distinguished explicitly.

These economics are an illustration, not a forecast.

The segment sizes, churn rates, recall and precision are measured values taken from the executed pipeline. The 30 % campaign success rate, the 50-unit offer cost and the twelve-month remaining-tenure assumption are not — the project holds no campaign-outcome data, so these three inputs cannot be validated from the data. The calculation shows the shape of the business case and the sensitivity of the return to each assumption; it should be re-run against observed campaign results before any budget decision rests on it.

14. Step-by-Step Configuration and Execution Guide

14.1 Prerequisites

1. Python 3.10 or later installed and available on the system path. The pipeline was verified on 3.13.13.
2. Approximately 500 MB of free disk space for the virtual environment, and a further 10 MB for generated artefacts.
3. The raw dataset present at data/Customer-Churn.csv relative to the working directory. Every path inside the scripts is relative to that directory.
4. At least 4 GB of available RAM. Phase 6 trains ensembles with 2 000 estimators and parallelises across all cores.

14.2 Step 1 — Prepare the environment

```
REM Open PowerShell or Command Prompt and move into the working directory.
REM Every script uses relative paths, so this directory must be current.

cd "C:\Users\user\Desktop\Customer churn prediction\churn_project\customer_churn"

REM Create and activate an isolated environment
python -m venv .venv
.venv\Scripts\activate

REM Install every dependency the six phases require
python -m pip install --upgrade pip
python -m pip install pandas numpy scikit-learn matplotlib seaborn xgboost joblib

REM Verify that all imports resolve, including xgboost
python -c "import pandas, numpy, sklearn, matplotlib, seaborn, xgboost, joblib; print('environment ok')"
```

Listing 4: Environment preparation. The final check must print "environment ok" before proceeding.

14.3 Step 2 — Confirm the folder layout

```
REM The data folder must already contain the raw file.
REM The charts and models folders are created automatically by the scripts,
REM but creating them ahead of time does no harm.

dir data\Customer-Churn.csv
if not exist charts mkdir charts
if not exist models mkdir models
```

Listing 5: Confirming the input file and preparing the output folders.

14.4 Step 3 — Execute the phases in order

The scripts must be run in numerical sequence, because each reads a file the previous one wrote. Run them one at a time and confirm each completes before starting the next.

```
python phase1_data_understanding.py
REM ~2 s   prints the data profile; writes nothing

python phase2_data_cleaning.py
REM ~3 s   writes data/Customer-Churn-Cleaned.csv

python phase3_eda.py
REM ~20 s  writes 9 charts into charts/
```

```
python phase4_feature_engineering.py
REM ~4 s   writes data/Customer-Churn-Model-Ready.csv

python phase5_customer_segmentation.py
REM ~30 s  writes data/Customer-Churn-Segmented.csv and 4 charts

python phase6_model_development.py
REM 15-45 min trains 5 models and runs a 150-fit randomized search;
REM          writes models/best_churn_model.joblib, the metric CSV and 3 charts
```

Listing 6: The complete execution sequence with indicative runtimes.

To run the whole pipeline unattended, chain the commands so that a failure in any phase halts the sequence rather than letting later phases run against stale input:

```
REM Command Prompt – && stops the chain on a non-zero exit code
python phase1_data_understanding.py && python phase2_data_cleaning.py && ^
python phase3_eda.py && python phase4_feature_engineering.py && ^
python phase5_customer_segmentation.py && python phase6_model_development.py

REM PowerShell equivalent
foreach ($s in @(("phase1_data_understanding", "phase2_data_cleaning", "phase3_eda",
                 "phase4_feature_engineering", "phase5_customer_segmentation",
                 "phase6_model_development"))) {
    python "$s.py"
    if ($LASTEXITCODE -ne 0) { Write-Host "FAILED: $s"; break }
}
```

Listing 7: Unattended execution of the full pipeline.

14.5 Step 4 — Verify the outputs

After phase	Expected artefact	Verification
2	data/Customer-Churn-Cleaned.csv	7 044 lines including the header; no blank cells
3	9 files matching charts/phase3_*.png	all nine present and non-empty
4	data/Customer-Churn-Model-Ready.csv	36 columns; all values numeric
5	data/Customer-Churn-Segmented.csv	25 columns including Cluster and CustomerSegment
5	4 files matching charts/phase5_*.png	elbow, silhouette, profiles, scatter
6	data/phase6_model_performance.csv	6 lines: header plus 5 models
6	models/best_churn_model.joblib	roughly 240 KB
6	3 files matching charts/phase6_*.png	ROC curves, confusion matrix, feature importance

Table 36: Artefacts to verify after each phase completes.

14.6 Step 5 — Use the trained model for prediction

The serialised object is a complete scikit-learn Pipeline, not a bare estimator. It contains the fitted ColumnTransformer as its first step, which means new data can be passed in raw — the same column names and types as the segmented dataset — with no manual preprocessing whatsoever.

```
import joblib
import pandas as pd

model = joblib.load("models/best_churn_model.joblib")

# New customers, with the same columns as Customer-Churn-Segmented.csv
# minus customerID, Churn, ChurnFlag and Cluster.
new_customers = pd.read_csv("data/new_customers.csv")

# Column 1 of predict_proba is the probability of the positive class (churn)
probabilities = model.predict_proba(new_customers)[1:, 1]
```

```

new_customers["ChurnProbability"] = probabilities
new_customers["RiskBand"] = pd.cut(
    probabilities,
    bins=[0.0, 0.30, 0.60, 1.0],
    labels=["Low", "Medium", "High"]
)

# Rank by revenue at risk rather than by probability alone
new_customers["RevenueAtRisk"] = (
    probabilities * new_customers["MonthlyCharges"] * 12
)

priority_list = new_customers.sort_values("RevenueAtRisk", ascending=False)
priority_list.head(200).to_csv("data/retention_priority_list.csv", index=False)

```

Listing 8: Scoring new customers and producing a revenue-ranked retention list.

14.7 Troubleshooting

Symptom	Cause	Resolution
<code>ModuleNotFoundError: No module named 'xgboost'</code>	XGBoost absent from the active interpreter	activate the virtual environment, then <code>pip install xgboost</code>
<code>FileNotFoundError: data/Customer-Churn-Cleaned.csv</code>	phase 3, 4 or 5 run before phase 2	run the phases in numerical order
<code>FileNotFoundError: data/Customer-Churn.csv</code>	wrong working directory	<code>cd</code> into <code>churn_project/customer_churn</code> first
<code>InconsistentVersionWarning</code> on load	model pickled under scikit-learn 1.8.0, loaded under 1.9.0	pin scikit-learn to 1.8.0, or re-run phase 6 to regenerate the file
Phase 6 appears to hang	150-fit randomized search in progress	expected; <code>verbose=1</code> prints progress. Reduce <code>n_iter</code> to shorten the search
Charts render but are empty	a non-interactive backend without a writable output folder	confirm <code>charts/</code> exists and is writable

Table 37: Common failure symptoms and their resolutions.

15. Observations, Limitations and Recommended Corrections

This section records what a close reading of the code found. The items are ordered by the severity of their effect on the reported results. Each states the observation, its consequence and the specific change that would resolve it.

15.1 The hold-out partition is 5 %, not the 20 % the code reports

The module declares `TEST_SIZE = 0.05`, which produces a hold-out set of 353 records. Two printed lines and one summary line nevertheless describe the split as 80/20:

```
TEST_SIZE = 0.05                                # <-- actual configuration

print("\n2. STRATIFIED TRAIN-TEST SPLIT")        # header says 80/20 in the summary
...
print("- Applied a stratified 80/20 train-test split.") # <-- contradicts the constant
```

The consequence is not a wrong number but a fragile one. Every metric in Table 28 is computed on 353 records, of which only 94 are churners. At that size the 95 % confidence interval on ROC-AUC is roughly ± 0.045 , which is more than twenty times the 0.0020 gap separating the top three models. This is the direct reason those three models cannot be meaningfully ranked against one another.

Recommended correction: set `TEST_SIZE = 0.20` to give a 1 409-record hold-out with roughly 374 churners, and correct the three print statements to read from the constant rather than hard-coding a percentage.

15.2 The phase 4 output is never used

Phase 4 constructs seven business features, one-hot encodes every categorical column, standardises every numeric column and writes a 36-column model-ready dataset. Phase 6 then opens Customer-Churn-Segmented.csv instead, which is the phase 5 output built from the cleaned file. The result is that ARPU, ChargeRatio, TenureBand, ContractRiskFlag, ValueTier and AutoPayFlag never reach any model. Only ServiceCount survives, and only because phase 5 recomputes it independently.

Six of the seven engineered features are therefore unused, and the phase 4 analysis in Section 9.4 — including the finding that churn peaks at exactly one add-on service — informs the report but not the model.

Recommended correction: merge the phase 4 features into the segmented dataset before phase 6 reads it, keyed on customerID. Because phase 6 builds its own scaling and encoding inside a leakage-safe ColumnTransformer, the raw engineered columns should be merged, not the pre-scaled versions — scaling them twice would be both redundant and, since phase 4 scales on the full dataset, a source of leakage.

15.3 The segment label carries information derived from the target

This is the most serious finding, because it affects the credibility of a reported result rather than merely wasting work.

Phase 5 assigns each cluster a business name using a rule that reads the cluster's observed churn rate:

```
churn_median = cluster_profile["ChurnRate"].median()

if tenure >= tenure_median and charges >= charge_median and churn < churn_median:
    segment_names[cluster_id] = "High-Value Loyal"
elif tenure >= tenure_median and charges >= charge_median and churn >= churn_median:
    segment_names[cluster_id] = "High-Value At-Risk"
```

```
elif tenure < tenure_median and churn >= churn_median:
    segment_names[cluster_id] = "New & Fragile"
```

The clustering itself is clean — K-Means sees only tenure, service count and monthly charges. The naming step is not: it computes each cluster's churn rate across the entire dataset, test rows included, and selects a label accordingly. Phase 6 then drops the numeric Cluster column but retains CustomerSegment as a categorical predictor, and the tuned model ranks it the second most important feature in the entire matrix at 0.150 gain.

The reported ROC-AUC of 0.8349 is therefore optimistic by an unknown margin. The effect is likely modest, since the label is coarse — four categories that a model could largely reconstruct from tenure and charges alone — but it is real, and it cannot be quantified without re-running the experiment.

Recommended correction, in order of preference. Either retain Cluster as the predictor and treat CustomerSegment as a presentation label only, which removes the leakage entirely at no cost to the model; or derive the segment names from the cluster centroids alone, using tenure and charges without reference to churn; or, if the churn-informed names are wanted, compute them on the training partition only and map them onto the test partition.

15.4 Two clusters received the same name

The naming rules classified clusters 0 and 3 identically, and the de-duplication branch appended a numeral, producing "New & Fragile" and "New & Fragile 2". The two are commercially very different: cluster 0 averages 29.58 in monthly charges with 24.19 % churn, while cluster 3 averages 77.58 with 47.37 % churn. A single shared root name obscures a distinction that the retention strategy in Section 13.2 depends on.

Recommended correction: extend the naming rule to test monthly charges as well as tenure and churn, so that cluster 0 becomes "New & Low-Value" and cluster 3 becomes "New & High-Value At-Risk".

15.5 Every model is fitted twice

```
for model_name, pipeline in final_models.items():
    pipeline.fit(X_train, y_train)          # first fit, inside the evaluation loop
    ...

best_model = final_models[best_model_name]
best_model.fit(X_train, y_train)          # second fit, on an already-fitted object
```

The best model is refitted after the evaluation loop has already fitted it. Because RANDOM_STATE is fixed the second fit is identical to the first, so no result changes — but on the 2 000-estimator ensembles the redundant work is measurable. Recommended correction: delete the second fit call.

15.6 An undocumented model artefact is present

The models folder contains enhanced_churn_model.joblib at 625 KB alongside the 238 KB best_churn_model.joblib. No script in the delivered folder writes a file of that name. It was presumably produced by a script that has since been removed or renamed, and its provenance, training data and performance are unknown.

Recommended correction: either restore the script that generates it, or remove the file. An unexplained model artefact in a deliverable folder is a reproducibility hazard, since a future reader has no way to tell which of the two files is authoritative.

15.7 Capabilities specified in the proposal that the code does not implement

The project proposal describes an eleven-component system. Six components are fully implemented; the remainder are not present in this codebase.

Component	Status	Evidence
Data cleaning and exploratory analysis	Implemented	phases 2 and 3
Customer segmentation	Implemented	phase 5, K-Means, 4 segments
Feature engineering	Implemented	phase 4, 7 features
Logistic Regression	Implemented	phase 6, ROC-AUC 0.8348
Random Forest and XGBoost	Implemented	phase 6, both present
Model comparison	Implemented	phase 6, 5 models across 5 metrics
Churn probability prediction	Implemented	<code>predict_proba</code> available on the saved pipeline
Explainable AI (SHAP)	Partial	gain importance only; no SHAP, no per-customer attribution
High-risk customer dashboard	Not implemented	no Streamlit or web module
Business recommendations engine	Not implemented	recommendations are analytical, not generated by code
Revenue-impact estimation	Not implemented	no module computes revenue at risk

Table 38: Implementation status of each component specified in the project proposal.

The four outstanding components are all downstream of a working model, which now exists and is serialised. A SHAP module would extend the existing importance chart to per-customer attribution; a Streamlit page would read the saved pipeline and render a ranked list; a business-impact module would apply the formulas already set out in Section 13.1. None requires the modelling work to be revisited.

15.8 Analytical limitations of the dataset itself

- **No temporal dimension.** The dataset is a single cross-sectional snapshot. There is no observation window, no prediction horizon and no event date, so the model answers "does this customer resemble one who has left" rather than "will this customer leave in the next ninety days". A production system needs the second question, and answering it requires time-stamped records.
- **No behavioural telemetry.** Support-ticket volume, network-quality incidents, usage trend and competitor pricing are all absent. These are typically the strongest churn predictors in telecommunications, and their absence is the most credible explanation for the 0.83-0.86 performance ceiling that this dataset consistently produces.
- **Probabilities are not calibrated.** Class weighting deliberately distorts the predicted probabilities in order to improve recall. A predicted 0.7 does not therefore mean a 70 % chance of churn, which is a problem the moment probability is multiplied by customer value to rank a call list. Platt scaling or isotonic regression on a held-out calibration set would correct this.
- **The decision threshold is fixed at 0.5.** The default cut-off is a mathematical convention, not a business choice. Since a missed churner costs far more than a false alarm, the optimal threshold is almost certainly lower. Sweeping it against the cost ratio in Section 13.4 would likely improve net benefit at no modelling cost at all.

16. Conclusion

The codebase delivers a complete and working analytics pipeline. Six sequential modules, totalling 1 709 lines, take a raw 7 043-record customer file through profiling, cleaning, exploratory analysis, feature construction, unsupervised segmentation and supervised modelling, producing three derived datasets, sixteen charts, a model-comparison table and a serialised prediction pipeline. Every artefact described in this report exists on disk and was read directly in preparing it.

16.1 What the implementation achieves

- **A validated dataset with no records lost.** Phase 2 repaired all eleven defective values using domain reasoning rather than statistical default, and retained all 7 043 records, so the class balance is preserved exactly and no selection bias is introduced.
- **Quantified churn drivers.** Contract type separates churn from 42.71 % to 2.83 %; the first six months carry a 52.94 % rate against a 26.54 % base; electronic-check payment reaches 45.29 %; and gender is confirmed as noise at a 0.76-point gap.
- **Four actionable segments found without the label.** K-Means on tenure, service count and monthly charges separated the base into groups whose churn rates span 4.44 % to 47.37 %, despite never being shown a churn value during fitting.
- **Five models trained and honestly compared.** The best hold-out ROC-AUC is 0.8349 at 0.7766 recall, competitive with published results on this dataset, with the full preprocessing pipeline serialised for reuse.
- **A leakage-safe modelling architecture.** All imputation, scaling and encoding sit inside a ColumnTransformer within each model pipeline, so fold statistics are learned only from training rows.

16.2 What the results actually support

The strongest claim the evidence supports is not the headline ROC-AUC. It is that three methodologically independent analyses — descriptive statistics, unsupervised clustering and a gradient-boosted ensemble — converged on the same five findings about what drives churn in this customer base. That convergence, set out in Table 32, is what makes the retention strategy in Section 13 defensible rather than merely plausible.

A weaker claim must be made about model ranking. The top three models separate by 0.0020 of ROC-AUC on a 353-record hold-out, which is far inside sampling noise. The correct statement is that Logistic Regression, Random Forest and tuned XGBoost perform equivalently on this data, and that Logistic Regression is preferable in practice because it achieves that performance in under a second and yields coefficients a retention team can read.

16.3 Priority actions

1. **Enlarge the hold-out partition.** Setting TEST_SIZE to 0.20 and correcting the three print statements would make the reported metrics stable enough to support a genuine model ranking. This is a one-line change with the largest effect on result quality of anything in this list.
2. **Remove the segment-label leakage.** Substituting the numeric Cluster column for the churn-derived CustomerSegment label in phase 6 would eliminate the leakage described in Section 15.3 and yield a trustworthy performance figure.
3. **Connect phase 4 to phase 6.** Merging the engineered features into the modelling matrix would put six currently unused business features to work, including the non-monotonic service-count signal that no existing model input captures.

4. **Build the four outstanding components.** SHAP attribution, the dashboard, the recommendation engine and the revenue-impact module are all downstream of the serialised model and require no change to the modelling work.
5. **Calibrate probabilities and tune the threshold.** Both are inexpensive, and both directly improve the quality of the revenue-ranked priority list that the business would actually act on.

With the first two corrections applied, the pipeline would move from a sound analytical demonstration to a result that could be defended in review. The remaining work extends the system outward toward deployment rather than repairing anything within it.

A. Appendix A — Configuration Constant Reference

Every configurable value in the codebase, grouped by the module that declares it. There is no shared configuration file, so a value used by more than one module must be changed in each.

A.1 Path constants

Module	Constant	Value
phase 1	FILE_PATH	"data/Customer-Churn.csv"
phase 2	INPUT_FILE	"data/Customer-Churn.csv"
phase 2	OUTPUT_FILE	"data/Customer-Churn-Cleaned.csv"
phase 3	INPUT_FILE	"data/Customer-Churn-Cleaned.csv"
phase 3	OUTPUT_FOLDER	"charts"
phase 4	INPUT_FILE	"data/Customer-Churn-Cleaned.csv"
phase 4	OUTPUT_FILE	"data/Customer-Churn-Model-Ready.csv"
phase 5	INPUT_FILE	"data/Customer-Churn-Cleaned.csv"
phase 5	OUTPUT_FILE	"data/Customer-Churn-Segmented.csv"
phase 5	CHART_FOLDER	"charts"
phase 6	INPUT_FILE	"data/Customer-Churn-Segmented.csv"
phase 6	CHART_FOLDER, MODEL_FOLDER	"charts", "models"

Table 39: Path constants declared across the six modules.

A.2 Analytical constants

Module	Constant	Value	Effect if changed
1, 2, 4	TARGET_COLUMN	"Churn"	renames the label column the pipeline reads
1, 2, 4	ID_COLUMN	"customerID"	changes which column is excluded from the model matrix
5, 6	RANDOM_STATE	42	a different seed changes cluster assignment, the split and every model result
5	OPTIMAL_K	4	alters the number of segments produced
6	TEST_SIZE	0.05	increasing to 0.20 would stabilise every reported metric
6	CV_FOLDS	5	more folds gives a lower-variance estimate at higher runtime cost

Table 40: Analytical constants and the effect of changing each.

A.3 Model hyper-parameters as configured

Model	Configuration as delivered
Logistic Regression	penalty="l2", solver="liblinear", class_weight="balanced", max_iter=2000
Random Forest	n_estimators=2000, max_depth=10, min_samples_split=5, min_samples_leaf=2, class_weight="balanced", n_jobs=-1
Support Vector Machine	kernel="rbf", C=1.0, gamma="scale", probability=True, class_weight="balanced"
XGBoost baseline	n_estimators=2000, learning_rate=0.10, max_depth=6, min_child_weight=1, subsample=0.85, colsample_bytree=0.85, scale_pos_weight computed from the training split
XGBoost tuned	RandomizedSearchCV over 10 parameters, n_iter=30, cv=5, scoring="roc_auc", refit=True — 150 fits in total
K-Means	n_clusters=4, n_init=10, random_state=42, fitted on three standardised dimensions

Table 41: Hyper-parameter configuration of every model in the codebase.

B. Appendix B — Complete Artefact Inventory

B.1 Datasets

File	Shape	Produced by	Consumed by
Customer-Churn.csv	7 043 × 21	supplied	phases 1, 2
Customer-Churn-Cleaned.csv	7 043 × 21	phase 2	phases 3, 4, 5
Customer-Churn-Model-Ready.csv	7 043 × 36	phase 4	nothing — see Section 15.2
Customer-Churn-Segmented.csv	7 043 × 25	phase 5	phase 6
phase6_model_performance.csv	5 × 6	phase 6	reporting (Table 28)

Table 42: Datasets produced by the pipeline and their consumers.

B.2 Generated charts

File	Phase	Content
phase3_univariate_analysis.png	3	four-panel distribution overview
phase3_churn_by_contract.png	3	churn rate by contract type
phase3_churn_by_payment_method.png	3	churn rate by payment method
phase3_churn_by_internet_service.png	3	churn rate by internet service
phase3_churn_by_senior_citizen.png	3	churn rate by senior status
phase3_churn_by_tech_support.png	3	churn rate by tech-support uptake
phase3_churn_by_tenure_band.png	3	cohort churn by tenure band
phase3_correlation_heatmap.png	3	numeric correlation matrix
phase3_tenure_monthlycharges_churn.png	3	behavioural scatter coloured by outcome
phase5_elbow_method.png	5	inertia against K
phase5_silhouette_scores.png	5	silhouette against K
phase5_cluster_profiles.png	5	segment metric comparison
phase5_customer_segments.png	5	segment scatter with service-count size
phase6_roc_curves.png	6	ROC overlay for all five models
phase6_confusion_matrix.png	6	confusion matrix of the best model
phase6_feature_importance.png	6	top twenty features by gain

Table 43: All sixteen charts generated by the pipeline.

B.3 Serialised models

File	Size	Contents	Provenance
best_churn_model.joblib	238 KB	fitted Pipeline: ColumnTransformer plus tuned XGBClassifier	written by phase 6
enhanced_churn_model.joblib	625 KB	unknown	no script in the folder produces this file — see Section 15.6

Table 44: Serialised model artefacts found in the models folder.

End of report.